算法设计与分析

Basic Maths Kit

阶乘 = $n!$ n个不同元素的全排列个数

stirling公式 = $n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n (1+\theta(\frac{1}{n}))$

(当n充分大时, $n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$

级数 =

1、多项式级数 = $\sum_1^n i = \frac{n(n+1)}{2} = \theta(n^2)$

$\sum_1^n i^2 = \frac{n(n+1)(2n+1)}{6} = \theta(n^3)$

$\sum_1^n i^k = \theta\left(\frac{1}{k+1} n^{k+1}\right) = \theta(n^{k+1})$

2、几何级数 = $\sum_0^k ar^i = \sum_1^{k+1} ar^{i-1} = a\left(\frac{r^{k+1}-1}{r-1}\right) = \theta(r^k)$

3、算术几何级数 = $\sum_1^k i2^i = (k-1)2^{k+1}+2$

4、调和级数 = $\sum_1^k \frac{1}{i} = \ln k + \gamma$ (欧拉常数) $+ \varepsilon_k$ ($\varepsilon_k \approx \frac{1}{2k}$)

5、斐波那契数列 = $F_n = \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right]$

记 $\phi = \frac{1+\sqrt{5}}{2}$, 则 $F_n = O(\phi^n)$

渐进表示 =

1、$f(n) = O(g(n))$ 给出函数的上界

$O(g(n)) = \{f(n) = \exists c>0, n_0>0$ 使 $\forall n \geq n_0$ 都有 $0 \leq f(n) \leq cg(n)\}$

$f(n) = O(g(n))$ iff $\lim_{n\to\infty} \frac{f(n)}{g(n)} = C < \infty$

2、$f(n) = o(g(n))$ 给出严格的上界, $g(n)$比$f(n)$大若干阶

$o(g(n)) = \{f(n) = \forall c>0$, 均$\exists n_0>0$ 使 $\forall n \geq n_0$ 都有 $0 \leq f(n) < cg(n)\}$

$f(n) = o(g(n))$ iff $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$

3、$f(n) = \Omega(g(n))$ 给出函数的下界

$\Omega(g(n)) = \{f(n) = \exists c>0, n_0>0$, 使 $\forall n \geq n_0$ 都满足 $0 \leq cg(n) \leq f(n)\}$

$f(n) = \Omega(g(n))$ iff $\lim_{n\to\infty} \frac{f(n)}{g(n)} = C > 0$

4. $f(n) = \omega(g(n))$ 给出更严格的下界，$f(n)$ 比 $g(n)$ 大若干所

$\omega(g(n)) = \{f(n) = \forall c>0$，均$\exists n_0>0$ 使$\forall n \geq n_0$ 都满足 $0 \leq cg(n) < f(n)\}$

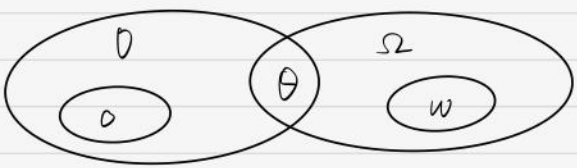
$f(n) = \omega(g(n))$ iff $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$

5. $f(n) = \Theta(g(n))$ 给出正好的上下界，$f(n)$ 和 $g(n)$ 同所

$\Theta(g(n)) = \{f(n) : \exists c_1>0, c_2>0, n_0>0$，使$\forall n \geq n_0$ 都满足 $0 \leq cg(n) \leq f(n) \leq c_2 g(n)\}$

$f(n) = \Theta(g(n))$ iff $\lim_{n\to\infty} \frac{f(n)}{g(n)} = c$，$(0<c<\infty)$

$f(n) = \Theta(g(n))$ iff $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$



集合论基本定理 = ZFC 公理体系

1. 外延公理 = 两个集合相等当且仅当它们包含相同的元素
2. 并集公理 = 对任意集合，其所有元素的并仍为集合
3. 幂集公理 = 集合的所有子集构成一个集合 (2^k)
4. 替换公理 = 若存在一个规则将集合中的每个元素映射到另一对象，则映射结果构成集合
5. 选择公理 = 对任意非空集合族，存在一个选择函数从每个集合中选取一个元素
6. 无穷公理 = 存在一个包含所有自然数的集合
7. 正则公理 = 不存在无限递降的集合序列，集合不能属于自身

数学归纳法 =

弱数学归纳法 = 设 P 为一个定义在 N 上的命题，若 =
① P(1) 为真，② $\forall k \in N$，$P(k) \to P(k+1)$
则对所有 $n \in N$，P(n) 为真

强数学归纳法 = 设 P 为一个定义在 N 上的命题，若 =
① P(1) 为真，② $\forall k \in N$，$P(1) \wedge \cdots \wedge P(k) \to P(k+1)$
则对所有 $n \in N$，P(n) 为真

良序定理 = 任意集合都可以被赋予一个良序关系
(该集合的任意非空子集都有一个最小元素)
设 P 为一个定义在 N 上的命题，若 =
命题不成立，则存在一个最小反例，导致矛盾

良序公理证明算法的正确性 = 将算法的执行过程映射到一个严格有序的数学结构上，
(贪心算法 / 动态规划 / 算法终止性 / 归纳法)

离散变量分布

算法分析中大多数状态转移可以认为是一次试验

求平均代价时可以引入随机性变量，使用期望 $E=\sum P(x_i)x_i$ 的形式求出平均代价

要区分是不是伯努利试性变量，既是不是"独立n次试验"、"有放回抽取"

伯努利实验：

1、伯努利分布（0-1分布）：$P(X=k)=p^k(1-p)^{n-k}$，$E(X)=np$

2、二项分布：n次独立伯努利试验中的成功次数：$P(X=k)=C_n^k p^k(1-p)^{n-k}$，$E(X)=np$

3、负二项分布：获得第r次成功所需的独立伯努利试验次数 $P(X=k)=C_{k-1}^{r-1}p^r(1-p)^{k-r}$，$E(X)=\frac{r}{p}$

3*、几何分布：首次成功所需的独立伯努利试验次数 $P(X=k)=p(1-p)^{k-1}$，$E(X)=\frac{1}{p}$

古典概型：

1、泊淞分布：固定区间内平均发生率λ，$P(X=k)=\frac{\lambda^k e^{-\lambda}}{k!}$ $E(X)=\lambda$

2、超几何分布：有限总体不放回抽取成功次数 $P(X=k)=\frac{C_M^m C_{N-M}^{n-m}}{C_N^n}$，$E(X)=\frac{M}{N}\cdot n$

例：对于一个大小为K的数组S[1..k]，使用顺序比较的算法UNIQUE判断其是否有重复的元素

即执行其至k(k-1)次比较来进行判断，平均时间复杂度是多少

每次比较时两个元素相等的概率为 $K\times(\frac{1}{k})^2=\frac{1}{k}$

直到第一次出现两个元素相等时算法结束，认为 $X\sim G(\frac{1}{k})$

则 $E(X)=k$，认为算法的平均时间复杂度为K

Master定理 (谁大取谁，相等乘logn)

Master定理：

$$T(n) = aT(\frac{n}{b}) + f(n)$$

n = 问题规模　　a 子问题个数

n/b = (假设每个子问题有相同大小) 子问题大小

f(n) = 拆解成子问题的开销

① 若 $\exists \varepsilon > 0$, $f(n) = O(n^{\log_b a - \varepsilon})$, then $T(n) = \Theta(n^{\log_b a})$

如果f(n)比 $n^{\log_b a}$ 的所小，则 $T(n) = \Theta(n^{\log_b a})$

② if $F(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$

如果f(n)和 $n^{\log_b a}$ 的所相等，则 $T(n) = \Theta(n^{\log_b a} (\log n))$

③ 若 $\exists \varepsilon > 0$, $f(n) = \Omega(n^{\log_b a + \varepsilon})$ and $\exists c < 1$, $\forall n \to \infty$ $af(\frac{n}{b}) \le cf(n)$, 则 $T(n) = \Theta(f(n))$

如果f(n)比 $n^{\log_b a}$ 的所大 (且是n的整数次方)

且 $\exists c < 1$ 使 $af(\frac{n}{b}) \le cf(n)$, 则 $T(n) = \Theta(f(n))$

证明：为方便推理，记 $f(n) = n^d$

0层 = 1个问题，$f(n) = n^d$，$f(n)$共 $= n^d$

1层 = a个问题，$f(n) = (\frac{n}{b})^d$，$f(n)$共 $= (\frac{a}{b^d}) \cdot n^d$

2层 = a^2个问题，$f(n) = (\frac{n}{b^2})^d$，$f(n)$共 $= (\frac{a}{b^d})^2 \cdot n^d$

...

k层 = a^k个问题，$f(n) = (\frac{n}{b^k})^d$，$f(n)$共 $= (\frac{a}{b^d})^k \cdot n^d$

...

$\lceil \log_b n \rceil$层 = $a^{\log_b n}$个问题，$f(n) = c \le 1$，$f(n)$共 $= a^{\lceil \log_b n \rceil} \cdot c$

因此，$T(n) = \sum_{i=0}^{\lceil \log_b n \rceil} (\frac{a}{b^d})^k n^d = n^d \frac{1 - (\frac{a}{b^d})^{\lceil \log_b n \rceil + 1}}{1 - \frac{a}{b^d}}$

当 $f(n) = n^d \log_x n$ 时

$f(n)$共 $= n^d \log n$

$f(n)$共 $= (\frac{a}{b^d}) n^d (\log_x n - \log_x b)$

$f(n)$共 $= (\frac{a}{b^d})^2 n^d (\log_x n - 2\log_x b)$

$f(n)$共 $= (\frac{a}{b^d})^k n^d (\log_x n - k\log_x b)$

显然，只有当 $f(n) = n^k$ 时才能使用等比数列求和，推出Master定理

① 当 $\frac{a}{b^d} = 1$ 时，$T(n) = (\lceil \log_b n \rceil + 1) n^d = \Theta(n^d \log n)$

② 当 $\frac{a}{b^d} < 1$ 时，n足够大时 $T(n) = \frac{1}{1 - \frac{a}{b^d}} n^d = \Theta(n^d)$

③ 当 $\frac{a}{b^d} > 1$ 时，n足够大时，$T(n) = (\frac{a}{b^d})^{\lceil \log_b n \rceil} n^d = c a^{\lceil \log_b n \rceil} = c n^{\log_b a} = \Theta(n^{\log_b a})$

另一种特例：$T(n)=aT(\frac{n}{b})+f(n)=aT(\frac{n}{b})+n^k\log n$

显然展开后得：$T(n)=\sum_{i=0}^{\lceil\log_b n\rceil}(\frac{a}{b^k})^i n^k(\log n-i\log b)$

$=\sum_{i=0}^{\lceil\log_b n\rceil}(\frac{a}{b^k})^i n^k\log n-\sum_{i=0}^{\lceil\log_b n\rceil}i(\frac{a}{b^k})^i n^k\log b$

当$\frac{a}{b^k}=1$，即$k=\log_b a$时，$T(n)=n^k\log n(\lceil\log_b n\rceil+1)-\frac{\lceil\log_b n\rceil}{2}(\lceil\log_b n\rceil+1)n^k\log b$

$\because \frac{1}{2}n^k\log n(\lceil\log_b n\rceil+1)=\Theta(n^k\log n\log_b n)=\Theta(f(n)\log n)$

当$\frac{a}{b^k}>0$且$\frac{a}{b^k}\neq 1$时：记$\frac{a}{b^k}=p$，$\lceil\log_b n\rceil=m$　$p^m=n^{\log_a b-k}$　$m\log b=\log n$

$T(n)=n^k\log n\frac{p^{m+1}-1}{p-1}-n^k\log b\frac{mp^{m+2}-(m+1)p^{m+1}+p}{p-1}$

当$n\to\infty$时，$m\to\infty$：

① 当$p>1$，即$\log_b a>k$时：（这两部分是平格相等的）

$T(n)=p^m n^k\log n+n^k\log b(p^{m+1}-mp^m)=\Theta(n^{\log_a b})$

② 当$0<p<1$时，即$\log_b a<k$时：

$T(n)=n^k\log n\frac{1}{1-p}-n^k\log b\frac{p}{(p-1)^2}=\Theta(n^k\log n)=\Theta(f(n))$

综上：$T(n)=\begin{cases}\Theta(f(n)), & \log_b a<k\\ \Theta(f(n)\log n), & \log_b a=k\\ \Theta(n^{\log_b a}) & \log_b a>k\end{cases}$　注意形式和Master定理相同

Akra-Bazzi 定理 = 用于处理 $f(n)$ 特殊的递归问题

引入多项式增长条件 = (看起来很高端，其实是"增长多项式"就可以使用)

定义 = $[b_k]$ 是一个 k 项数列且 $0<b_i<1$，若 $\exists c_1, c_2>0$ 对 $\forall x\geq 1$，$1\leq i\leq k$，$u\in[b_i x, x]$ 均有 $c_1 g(x)\leq g(u)\leq c_2 g(x)$，则称 $g(x)$ 满足多项式增长条件

定义2 = $\exists C>0, k>0$，使对 $\forall$ 足够大 x 和 $y\geq x/C$，满足 =

$$f(y)\leq C\left(1+\frac{y-x}{x}\right)^k f(x)$$

简化为 =

$$f(y)\leq C\left(\frac{y}{x}\right)^k f(x),\ 当 y\geq x/C$$

再变形为 =

$$f(y)/f(x)\leq C\left(\frac{y}{x}\right)^k,\ 当 y\geq x/C$$

以此，$f(n)$ 的增长速率被 y^k 所控制，保证可积 (防震荡，防激增)

同时提供比例约束 = $y=2x$ 时，$C\geq\frac{1}{2}$，$f(2x)/f(x)\leq 2^k\times C$

保证 $n\to\infty$ 时，$f(x)$ 的主导式是多项式形式

更简单的定义 = 若 $\exists C>0$，使得 $|g(x)|=O(x^c)$，则 $g(x)$ 肯定满足多项式增长条件

Akra-Bazzi 定理 = 用于解决不均匀 / $f(n)$ 复杂的递归问题

$$T(n)=\sum_{i=1}^{k} a_i T(b_i n+h_i(n))+f(n)$$

① $a_i>0$，$b_i\in(0,1)$，$h_i(n)=o(n)$ (即 $n\to\infty$ 时 $h_i(n)/n=0$)

② $f(n)$ 满足多项式收敛条件

③ 存在唯一的实数 p，使得 $\sum_{i=1}^{k} a_i b_i^p=1$

则 $T(n)=\Theta\left(n^p\left(1+\int_1^n \frac{f(u)}{u^{p+1}}du\right)\right)$

例1：归并排序 $T(n)=2T(\frac{n}{2})+n$

$\Sigma a_i b_i^p = 2\times(\frac{1}{2})^p=1$ 得 $p=1$

$T(n)=\Theta(n(1+\int_1^n \frac{n}{n^2}du))=\Theta(n+n\ln n)=\Theta(n\log n)$

例2：$T(n)=2T(\frac{n}{4})+3T(\frac{n}{6})+n\log n$

$\Sigma a_i b_i^p = 2(\frac{1}{4})^p+3(\frac{1}{6})^p=1$ 得 $p=1$

$T(n)=\Theta(n(1+\int_1^n \frac{u\ln u}{u^2}du))=\Theta(n(1+\frac{1}{2}\ln^2 n))=\Theta(n\log^2 n)$

例3：检验 $T(n)=aT(\frac{n}{b})+n\log n$（取 $k=1$）

$$T(n)=\begin{cases}\Theta(n^{\log_b a}) & ,\log_b a>1\\ \Theta(f(n)\log n)=\Theta(n\log^2 n) & ,\log_b a=1\\ \Theta(f(n))=\Theta(n\log n) & ,\log_b a<1\end{cases}$$

① $\log_b a=k=1$ 取 $a=b=2$

$\Sigma a_i b_i^p = 2(\frac{1}{2})^p=1$ 得 $p=1$

$T(n)=\Theta(n(1+\int_1^n \frac{u\log u}{u^2}du))=\Theta(n\log^2 n)$ ✓

② $\log_b a<k$ 取 $a=2$ $b=4$，$\log_b a=\frac{1}{2}$

$\Sigma a_i b_i^p = 2(\frac{1}{4})^p=1$ 得 $p=\frac{1}{2}$

$T(n)=\Theta(n^{\frac{1}{2}}(1+\int_1^n \frac{u\log u}{u^{3/2}}du))=\Theta(n^{\frac{1}{2}}(1+2\ln n\sqrt{n}-4\sqrt{n}+4))=\Theta(n\log n)$ ✓

③ $\log_b a>k$，取 $a=4$ $b=2$，$\log_b a=2$

$\Sigma a_i b_i^p = 4(\frac{1}{2})^p=1$ 得 $p=2$

$T(n)=\Theta(n^2(1+\int_1^n \frac{u\log u}{u^3}du))=\Theta(n^2(1-\frac{\ln n}{n}-\frac{1}{n}+1))=\Theta(n^2)$ ✓

欧几里得算法(辗转相除法)

算法核心: $gcd(a,b) = gcd(b, a\%b)$

证明: 设a和b的最大公因数为C, $c = gcd(a,b)$, 设 $a = xc$ $b = yc$ 则有x和y互质

设 $g = a\%b = a - ib = (x - iy)c$ 其中 $i = \lfloor a/b \rfloor$

欲证 $gcd(a,b) = gcd(b, a\%b) = c$, 即证 $x - iy$ 与 y 互质:

设 $x - iy$ 与 y 不互质, 则 $x - iy$ 与 y 存在最大公约数 $k (k>1)$, 使 $x - iy = nk$, $y = mk$ 且 n, m 互质

则 $x = (n + im)k$ $y = mk$ 与前面的假设矛盾 $(x, y$ 互质$)$, 因此 $x - iy$ 与 y 互质

综上, 得证 $gcd(a,b) = gcd(b, a\%b)$

复杂度分析:

① 若 $a = b$, 则 $gcd(a,b) = a = b$

② 若 $a < b$, 则 $gcd(a,b) = gcd(b, a\%b) = gcd(b, a)$

③ 若 $a > b$, 则 $gcd(a,b) = gcd(b, a\%b)$

- ① $a = 2b$ 时, $gcd(b, a\%b) = b$
- ② $a < 2b$ 时, 设 $a = 2k$, $b = k + m$

  $gcd(b, a\%b) = gcd(k+m, k-m) = gcd(k-m, (k+m)\%(k-m))$

  因此两次操作后数据量最差会变为原来的一半
- ③ $a > 2b$ 时, 设 $a = 2k$, $b = k - m$

  $gcd(b, a\%b) = gcd(k-m, (2k)\%(k-m))$

  一次操作后数据量就变为原来的一半

因此关键操作的复杂度为 $2\log N$, $T(n) \in O(\log N)$

展开后根据数学特征分析算法时间复杂度的典型, 类似的方法有递归树法等

代入法：用于处理结构特殊（如T(n-1)，非树型递归）的递归问题和多变量递归问题

代入法的正确性依赖于数学归纳法：（必须手格处理非递归项f(n)，若验证失败，需调整猜测的解）

1. 归纳基：验证解在初始值的正确性
2. 归纳步：假设对 $\forall m<n$ 成立，则n也成立
3. 闭合性：通过调整常数c，确保递归展开后的表达式的正确性

例：归并排序：$T(n)=2T(n/2)+O(n)$

猜测解：$T(n)\in O(n\log n)$　　假设基：$T(1)=1$

假设：对所有 $m<n$，均有 $T(m)\le cm\log m$　下面证 $T(n)\le cn\log n$

代入递归式：$T(n)=2T(\frac{n}{2})+O(n)\le 2\cdot c\frac{n}{2}\log\frac{n}{2}+O(n)=cn\log n-cn+O(n)\le cn\log n-(c-k)n$

当 $c\ge k$ 时，$T(n)\le cn\log n$，

综上，猜测成立，$T(n)=O(n\log n)$

例2：主定理处理不了的情况：$T(n)=2T(n/2)+n\log n$　$\log_b a=1=k$，$T(n)=\Theta(n\log^2 n)$

猜测解：$T(n)\in O(n\log^2 n)$

假设：对所有 $m<n$，都有 $T(m)\le cm\log^2 m$

代入递归式：$T(n)=2T(\frac{n}{2})+n\log n\le 2\frac{cn}{2}\log^2\frac{n}{2}+n\log n=cn(\log n-1)^2+n\log n$

$=cn\log^2 n-(2cn\log n-n\log n-cn)$

当 $c\ge 1$ 时，$T(n)\le cn\log^2 n$

综上，猜测成立，$T(n)\in O(n\log^2 n)$

例3：$T(n)=2T(n/2)+1$　$\log_b a=1>k=0$

猜测解：$T(n)\in O(n)$

假设：对所有 $m<n$，都有 $T(m)\le cm$

代入递归式：$T(n)=2T(\frac{n}{2})+1\le 2\frac{cn}{2}+1=cn+1$　$cn+1$ 不可能 $\le cn$

假设：对所有 $m<n$，都有 $T(m)\le cm-d$（当原假设不成立时，使用更强的假设）

代入递归式：$T(n)=2T(\frac{n}{2})+1\le 2(c\frac{n}{2}-d)+1=cn-2d+1$

当 $c>0$，$d\ge 1$ 时，$T(n)\le cn-d$

综上，假设成立，$T(n)\in O(n)$

例4： $T(n)=2T(\lfloor\sqrt{n}\rfloor)+\log n$

形式上有次方项，可以令 $n=2^m$ 后设 $S(m)=T(2^m)$

$S(m)=T(2^m)=2T(2^{\frac{m}{2}})+m=2S(\frac{m}{2})+m$ 得 $S(m)=O(m\log m)$

$n=\log_2 m$，$T(n)=O(\log n\times\log(\log n))$

例5，$T(n)=T(n/3)+T(2n/3)+O(n)$

猜测解 $T(n)=O(n\log n)$，设 $C_1>0$，当 $n\geq n_1$ 时，$T(n)\leq T(\frac{n}{3})+T(\frac{2n}{3})+C_1n$

假设 $\exists C_2>0, n_2>0$，使 $\forall n\geq n_2$，有 $T(n)\leq C_2n\log n$

则 $T(n)\leq T(\frac{n}{3})+T(\frac{2n}{3})+C_1n\leq\frac{C_2n\log\frac{n}{3}}{3}+\frac{2C_2n\log\frac{2n}{3}}{3}+C_1n=C_2n\log n-(C_2n\log_3-\frac{2}{3}C_2n\log 2-C_1n)$

可知，当 $C_2>\frac{C_1}{\log_3-\frac{2}{3}\log 2}$，$n\geq n_1$ 时，有 $T(n)\leq C_2n\log n$ $T(n)=O(n\log n)$

递归树展开法

更细的过程见前面主定理

对于 $T(n)=aT(n/b)+f(n)$ 展开后得

记 $m=\log_b n$，$T(n)=a^mT(1)+a^{m-1}f(\frac{n}{b^{m-1}})+a^{m-2}f(\frac{n}{b^{m-2}})+\cdots+f(n)$

假设 $f(n)=O(n^k)\leq cn^k$

$T(n)=a^mT(1)+(\frac{a}{b^k})^{m-1}cn^k+(\frac{a}{b^k})^{m-2}cn^k+\cdots+cn^k$

$=a^mT(1)+(1-(\frac{a}{b^k})^m)/(1-\frac{a}{b^k})cn^k$

$=\begin{cases}c/(1-\frac{a}{b^k})(n^k-n^{\log_b a})+n^{\log_b a}T(1), & k\neq\log_b a\\ cn^k\log_b n+n^k, & k=\log_b a\end{cases}$

多变量递归

多变量递归不只是对算法的复杂度分析，还是对算法进行优化，不同的计算方法会有不同的结果

1. 递归树打展法：基本上是强行展开，且不考虑相同的行为重复执行
2. 动态规划状态表法：将递归转化为表格填充，计算每个状态的计算次数

（标准的动态规划流程）

定义状态维度：如二维表格 dp[n][m]

分析状态依赖：确定每个状态的前驱状态数

计算总状态数：时间复杂度 = 状态数 × 单状态计算时间

3. 代入法（数学归纳法）：要重点处理边界条件，统一展开
4. 主定理及 Akra-Bazzi 定理：若递归式可以分离变量，尝试合并应用定理

例1：编辑距离的递归实现：

计算将字符串A[长m]转换为字符串B[长n]的最小增删改次数：

$$T(m,n) = T(m-1,n) + T(m,n-1) + T(m-1,n-1) + O(1)$$

A删除　A增加　A修改/跳过　单次操作

1. 递归树打展：3叉树，树高 $\min(m,n)$，因此复杂度 $O(3^{\min(m,n)})$，非常逆天
2. 动态规划：一共有 $m \times n$ 种状态，每个状态只会进入一次，每次执行的复杂度为 $O(1)$

因此一共有 $m \times n \times O(1) = O(mn)$ 的复杂度

例2：资源分配/选择寻路问题：

从坐标(0,0)到(a,b)的路径数，每次只能向左或向上

$$T(a,b) = T(a-1,b) + T(a,b-1) + O(1)$$

1. 递归树打展：2叉树，树高 $m+n$，共 $O(2^{m+n})$ 复杂度
2. 动态规划：一共有 $a \times b$ 种状态，共 $O(mn)$ 复杂度
3. 变量替换法：设 $k = m+n$，则 $T(k) = 2T(k-1) + O(1)$，共 $O(2^k)$ 复杂度

例3：二维分治算法：将 $m \times n$ 的矩阵分为4个 $\frac{m}{2} \times \frac{n}{2}$ 的子块，合并代价为 $O(mn)$

$$T(m,n) = 4T(m/2, n/2) + O(mn)$$

1. 递归树：树高 $k = \log \min(m,n)$，树一层节点数 4^k，一层开销 $4^k \times O(\frac{m}{2^k} \times \frac{n}{2^k}) = O(mn)$

总时间复杂度 $T(m,n) = O(mn) \cdot \log \min(m,n) = O(mn \log \min(m,n))$

2、对称性假设：设 $m=n$，则 $T(n,n)=4T(\frac{n}{2},\frac{n}{2})+O(n^2)$

记 $B(n)=T(n,n)$，则 $B(n)=4B(\frac{n}{2})+O(n^2)$，易知 $B(n)=O(n^2\log n)$

由迭代分析知 $T(m,n)=O(mn\log\min(m,n))$

3、代入法：猜测解：$T(n)=O(mn\log\min(m,n))$

假设：对于所有 $m<M$、$n<N$，都有 $T(m,n)\le cmn\log\min(m,n)$

$T(M,N)=4T(\frac{M}{2},\frac{N}{2})+O(MN)\le 4c\frac{MN}{4}\log\min(\frac{M}{2},\frac{N}{2})+kMN$

$=cMN\log\min(M,N)-(cMN-kMN)$

当 $c>k>0$ 时，$T(M,N)\le cMN\log\min(M,N)$

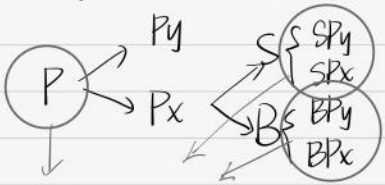
综上 $T(M,N)=O(MN\log\min(M,N))$

二维最近点对问题

Divide =

对于一个乱序的点集P，我们分别对x和y进行排序，得到Px和Py

之后将Px通过中位数进行划分，得到SPx、BPx，并"根据y重排"得到SPy、SPx



"根据y重排"是顺序便历Py，通过匹配x来得到对应数组，这样可以将O(nlogn)减少到O(n)

原问题被划分为3个完全相同的子问题，最终只有N小于等于3个点时，可以直接比较每个点对得出答案

Merge =

对于当前的点集，会得到SP的最小点对$\{s_1, s_2\}$，距离δ_1，BP的最小点对$\{b_1, b_2\}$，距离δ_2

此外，还有可能会出现$s_0 \in SP$，$b_0 \in BP$，$\{s_0, b_0\}$是最小点对，距离δ_3

我们首先令$\delta = \min\{\delta_1, \delta_2\}$，现在只要比较可能的$\delta_3$和$\delta$就可以

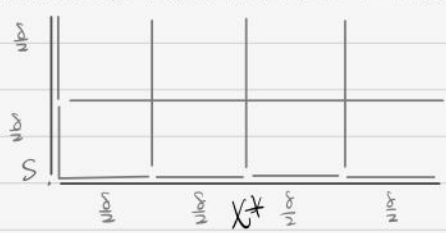
草率的方法：对中位数点x^*，向左右各延长δ，只比较这个区域内的点，但这不是稳定的剪枝方法

但可能所有的点都集中在这个区域里，则仍然没能减少算法复杂度上限，也就是$O(n^2)$

现在我们希望对于检查某一点n，可以构造一个区域N，使N中的点个数有限，N包含了S²的范围

因此我们要使用Py，并给出N的定义 = 对于N是在x^*左右延长δ的区域M中的一点

假设$n \in S$，则N是以n为起点，x轴右向长2δ，y轴右向长δ的区域，并划分为$\frac{\delta}{2} \times \frac{\delta}{2}$的8个区域(box)



一个box的最长距离为$\frac{\delta}{2}$，因此，一个box中最多只有1个点

因此对于点S，只要检查M中比它稍大的7个点即可

这7个点一定包含N区域中的所有点

因此使用Py顺序遍历7个点即可，上限为$7n = O(n)$

综上：$T(n) = \begin{cases} 2T(n/2) + O(n) & n > 3 \\ O(1) & n \le 3 \end{cases}$ 易知$T(n) = O(n\log n)$

加上初始排序$O(n\log n)$，仍为$O(n\log n)$

三维最近点对问题

Divide =

虽然多个z轴的信息，但是我们仍然可以只选两个方向排序，这里取x、z，得到P_x、P_z

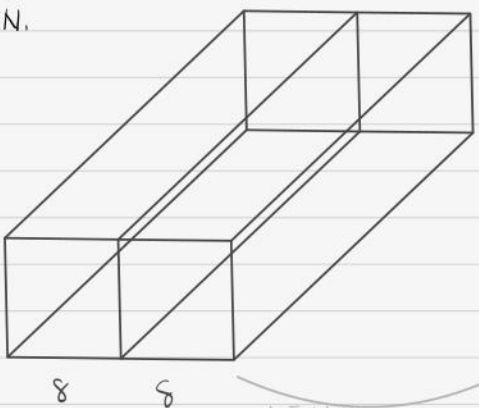
并划分出对于x中位数x^*的两个区域 SP_x & SP_z，BP_x & BP_z

Merge =

首先我们还是使用x^*左右δ区域做一个区域M，但是现在只通过x-z平面的鸽笼原理是不起效的

现在我们需要找一个x-y-z的N.





目前给出的box为4-2-2

每个box的边长为$\frac{\delta}{2}$

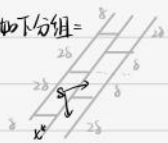
一共有4×2×2=16个box

其实是2×8×2=32个box

我们如何明确在y的划分呢？

对于任意S_y，其区域为$[S_y-\delta, S_y+\delta]$

这个限制是双向的，我们也不能对于每个S都遍历一次P_z，故我们可以构造如下分组=



本分组的S会对应对右两个相邻分组，宽为4δ

所以我们要用O(n)时间将P_z划分成P_{yl}（桶形式），之后在对应桶各找P_z从小到大16个，就可以做到对32n=O(n)以内找到δ_3.

$$T(n)=\begin{cases}2T(n/2)+O(n), & n>3\\ O(1), & n\le 3\end{cases}$$

易知$T(n)=O(n\log n)$ 即使前面排x、y、z也可以$O(n\log n)$

故算法时间复杂度为$O(n\log n)$

平摊分析

一、聚合分析：能够知道T(n)上界时（T(n)=O(g(n))），用$O(\frac{g(n)}{n})$表示平摊代价

二、会计方法：平摊代价 = 实际代价 + 存储信用 （所有操作的总信用应始终非负）

三、势能方法：定义一个势能函数Φ(D)，将当前状态D映射到一个非负实数。（相当于保证了所有操作总信用非负）

平摊代价 = 实际代价 + 势能变化 $(\Phi(D')-\Phi(D))$

会计方法一般针对单个操作/元素，势能方法针对的是一种状态/数据结构的特点。

例1：动态数组扩容

1、聚合分析：$T(n)=\sum_{k=0}^{\lfloor \log n \rfloor} 2^k + n \le 3n-1 = O(n)$　　$\frac{T(n)}{n}=\frac{O(n)}{n}=O(1)$

2、会计方法：每次扩容时：



假设每个元素进入时的"信用"为k

上面这一部分是从前面的数据移来的，信用要减1，故信用+=1

下面这一部分是新加入的，但是为了补足之，信用要再减1，信用+=1

所以每次扩容时信用要减2

因此要求信用-2≥1（放入代价），故信用≥3。

3、势能方法：设计Φ(D) = 2×当前的元素数 − 数据容量　(assert(Φ(D)) ≥ 0)

对于不扩容的情况，设当前元素为k，容量为c

$\Delta\Phi(D)=(2(k+1)-c)-(2k-c)=2$

平摊代价 = 1 + ΔΦ(D) = 3

对于要扩容的情况，设当前元素为k，此时的容量为k

$\Delta\Phi(D)=(2(k+1)-2k)-(2k-k)=2-k$

平摊代价 = 1 + ΔΦ(D) = 3 − k < 3

例2、计数器进位问题：分析一个n位二进制计数器

1、聚合分析：最低位翻转2^n次，次低位翻转2^{n-1}次，…，最高位翻转1次

$\sum_{i=1}^{n} 2^i = 2^{n+1}-1$　平摊代价 $\frac{2^{n+1}-1}{2^n}=2=O(1)$

2、会计方法：每次计数器加1，最低位会翻转，我们假设最低位平摊代价为k

①最低位是0，操作后总信用为k-1，最低位变为1

②最低位是1，操作后总信用为2k-2，最低位变为0

但此时进了一位，我们希望进的一位也有初始状态下的信用，即k-1

k-1是支付过状态改变后的信用，因此我们要从总信用-(k-1)-1

(2k-2)-(k-1)-1 = k-2 ≥ 0，因此取k=2，故平摊代价为2

3. 势能方法：记当前有效位为 b，从最低位到最高位有连续 d 个 1，b以内有 C 个 1，记势能为 b−C

① 不发生进位：$\Phi(D)=b-C$　$\Phi(D')=b-(C-d+1)$　$\Delta\Phi(D)=d-1$

② 发生进位：$b=C=d$，$\Phi(D)=1$　$\Phi(D')=b+1-1=b$　$\Delta\Phi(D)=d$

故平摊代价 $\le d+1$

观察到 d 满足如图所示的二叉树的前序排列，根据树的特性，设有 h 层树高为 h，



则树上所有点的和 $T(h)=2T(h-1)+2^{h-2}$，$T(1)=0$。

得 $T(h)=(h-1)2^{h-2}$ 对第 m 个点，其所在的子树高 $\lfloor\log_2 m\rfloor$

所在子树高 $\lfloor\log_2 m\rfloor+1$，由糖水不等式易知，平均代价 C

$$C\le\frac{T(\lfloor\log_2 m\rfloor)+2^{\lfloor\log_2 m\rfloor-1}}{m}<\frac{1}{2}\frac{\log m}{m}+\frac{1}{2}<1$$

故平摊代价 ≤ 2

指标随机变量

划分原子事件 → 定义指标变量 → 计算概率 $P(I_A)$，$E(I_A)=P(I_A)$ → 总期望 $E(X)=E(\sum I_A)=\sum E(I_A)$

应用：分析快速排序平均时间复杂度

划分/定义：X_{ij} 表示 $A[i]$ 和 $A[j]$ 是否直接比较，则 $E(X)=\sum_{i=1}^{n}\sum_{j=i+1}^{n}E(X_{ij})$

计算概率：对于任意 $i<j$，可以将整个数组划分为5部分（在这里 X_i 到 X_n 已排序）

$X_1 \cdots X_{i-1} \mid X_i \mid X_{i+1} \cdots X_{j-1} \mid X_j \mid X_{j+1} \cdots X_n$

记为①②③④⑤，pivot可能是这5部分中的任意一点。

当pivot在①和③时，X_i 和 X_j 将被划分在同一组，本次划分无影响

当pivot在③时，X_i 和 X_j 将被分到两组，永远不会发生比较

当pivot在②④时，X_i 和 X_j 将发生直接比较

算法的终止性是显然的，只有②④③三部情况对 X_{ij} 有影响

因此 $X_{ij}=\frac{|②|+|④|}{|②|+|③|+|④|}=\frac{2}{j-i+1}$

计算总期望：$\sum_{i=1}^{n}\sum_{j=i+1}^{n}X_{ij}=\sum_{i=1}^{n}\sum_{j=i+1}^{n}\frac{2}{j-i+1}=\sum_{i=1}^{n}\sum_{k=1}^{n-i}\frac{2}{k+1}=\sum_{i=1}^{n}\sum_{k=1}^{n}\frac{2}{k}=O(n\log n)$

应用2：雇佣问题变：给定一个数组 $A[1..n]$，取 $A[1..n]$ 中比前面所有元素更大的元素，求总雇佣次数期望

划分/定义：X_i 表示第 i 名候选人被雇佣，则 $E(X)=\sum E(X_i)$

计算概率：$X_i=1$ 则他是前 i 个中的最优，概率为 $\frac{1}{i}$

计算总期望：$E(X)=\sum E(X_i)=\sum\frac{1}{i}=O(\log n)$

应用3：分析快速选择算法的平均时间复杂度

划分/定义：$X_k=I\{$小于或等于其值元素的元素个数为 $k\}$

计算概率：显然有 $E(X_k)=\frac{1}{n}$

计算总期望：$T(n)\le\sum_{k=1}^{n}X_k(T(\max(k-1,n-k))+O(n))$

$E(T(n))\le\sum_{k=1}^{n}E(X_k)E(T(\max(k-1,n-k)))+E(X_k)O(n)$

$=n\,O(1)+\frac{2}{n}\sum_{k=\frac{n}{2}}^{n}E(T(k))=\frac{2}{n}\sum_{k=\frac{n}{2}}^{n}E(T(k))+O(n)$

$\therefore E(T(n))\le\frac{2}{n}\sum_{k=\frac{n}{2}}^{n}E(T(k))+O(n)$

假设存在某个 $c>0$，$n_0>0$，使 $\forall n>n_0$ 有 $E(T(n))\le cn$

$E(T(n))\le\frac{2}{n}\sum_{k=\frac{n}{2}}^{n}E(T(k))+O(n)\le\frac{2}{n}\sum_{k=\frac{n}{2}}^{n}ck+O(n)$

$=\frac{3n+1}{4}+O(n)\le\frac{3n+1}{4}+c_2n$

当 $c\ge c_2+\frac{3}{4}$ 时可满足此条件，故 $E(T(n))\in O(n)$

R. Morris概率计数器

（概率计数） 利用一个 b 位的计数器，我们一般只能计数到 2^b-1。而用 R. Morris 的概率计数法，我们可以计数到一个大得多的值，代价是精度有所损失。

对 $i=0, 1, \cdots, 2^b-1$，令计数器值 i 表示 n_i 的计数，其中 n_i 构成了一个非负的递增序列。假设计数器初值为 0，表示计数 $n_0=0$。INCREMENT 运算单元工作在一个计数器上，它以概率的方式包含值 i。如果 $i=2^b-1$，则该运算单元报告溢出错误；否则，INCREMENT 运算单元以概率 $1/(n_{i+1}-n_i)$ 把计数器增加 1，以概率 $1-1/(n_{i+1}-n_i)$ 保持计数器不变。

对所有的 $i\geqslant 0$，若选择 $n_i=i$，此计数器就是一个普通的计数器。若选择 $n_i=2^{i-1}(i>0)$，或者 $n_i=F_i$（第 i 个斐波那契数，参见 3.2 节），则会出现更多有趣的情形。

对于这个问题，假设 n_{2^b-1} 已足够大，发生一个溢出错误的概率可以忽略。

a. 请说明在执行 n 次 INCREMENT 操作后，计数器所表示的数期望值正好是 n。$E(n_X)=n_{E(X)}=X$

b. 分析计数器表示的计数的方差依赖于 n_i 序列。我们来看一个简单情形：对所有 $i\geqslant 0$，$n_i=100i$。在执行了 n 次 INCREMENT 操作后，请估计计数器所表示数的方差。

a. $X=0$ 时，执行1次INCREMENT操作，X 有 $\frac{1}{n_1}$ 的概率为1，有 $1-\frac{1}{n_1}$ 的概率为0

则 $E(n_1)=\frac{1}{n_1}\times n_1+(1-\frac{1}{n_1})\times 0=1$ 故 $n=1$ 时成立

假设当 $n\leq k-1$ 时均成立，对 $n=k$，执行一次INCREMENT操作，X 有 $\frac{1}{n_{k+1}-n_k}$ 的概率加1

则 $E(n_{k+1})=E(\frac{1}{n_{k+1}-n_k}n_{k+1}+(1-\frac{1}{n_{k+1}-n_k})n_k)=E(n_k)+1$ $\because E(n_k)=k$ 故 $E(n_{k+1})=k+1$ 成立

由归纳得计数器的期望值正好为 n

b. 由a、$E(n_k)=E(n_{k-1}+1)=k$，从 $X=k$ 到 $X=k+1$ 服从几何分布，$p=\frac{1}{n_{k+1}-n_k}$，$D(X)=\frac{1-p}{p^2}$

故 $D(X=k)=\sum_{i=0}^{k-1}D(i)$，$\because n_i=100i$ 故认为 $k=100$ $D(i)=9900$

$D(X=k)=\sum_{i=0}^{n-1}D(i)=9900n$

计数排序 = 空间换时间的算法 期望 $O(n+k)$

对数组 $A[1..n]$ 进行排序，且每个输入都在 $[1..k]$ 区间内

构建数组 $B[1..k]$，遍历 A，$B[A[i]]+=1$。最后遍历 B，$B[i]$ 表示数字 i 的个数。

算法时间复杂度 $O(n+k)$，空间复杂度 $O(k)$

计数排序拓展 = 基数排序 期望 $\Theta(n)$

给定 n 个 d 位数，每一位有 k 个可能的取值，若使用 $\Theta(n+k)$ 的排序算法（计数排序），则可以在 $\Theta(d(n+k))$ 排序

拓展 = 给定任何正整数 $r\le d$，则可以在 $\Theta((d/r)(n+2^r))$ 的时间内排好序

显然有 $d=\log n$，则取 $r=\log n$ 可使其复杂度为 $\Theta(n)$

当 $|B|<k$ 时 = 桶排序 期望 $\Theta(n)$

$T(n)=\Theta(n)+\sum_{i=0}^{n-1}O(n_i^2)$，其中 n_i 表示桶 $B[i]$ 中元素个数

$E[T(n)]=E[\Theta(n)+\sum_{i=0}^{n-1}O(n_i^2)]=\Theta(n)+\sum_{i=0}^{n-1}O(E[n_i^2])$

每个元素等概率地落入任意一个桶中，每个桶 i 具有相同的期望值 $E[n_i^2]$

定义示性变量 $X_{ij}=I\{A[j]$ 落入桶 $i\}$

$$E[n_i^2]=E[(\sum_{j=1}^{n}X_{ij})^2]=E[\sum_{j=1}^{n}\sum_{k=1}^{n}X_{ij}X_{kj}]=E[\sum_{j=1}^{n}X_{ij}^2]+E[\sum_{j=1}^{n}\sum_{\substack{k=1\\ k\neq j}}^{n}X_{ij}X_{kj}]$$

$$=\sum_{j=1}^{n}E(X_{ij}^2)+\sum_{1\le i\le n}\sum_{\substack{1\le k\le n\\ k\neq j}}E[X_{ij}X_{ik}]$$

显然有 $E(X_{ij}^2)=\frac{1}{n}\times 1^2+(1-\frac{1}{n})\times 0^2=\frac{1}{n}$

$E(X_{ij}X_{ik})=E(X_{ij})E(X_{ik})=\frac{1}{n^2}$

$E[n_i^2]=n\times\frac{1}{n}+n(n-1)\frac{1}{n^2}=1+\frac{n-1}{n}=2-\frac{1}{n}$

故 $E[T(n)]=\Theta(n)+n\,O(2-\frac{1}{n})=\Theta(n)$

分治法：

基本流程：划分、解决、合并　大多数情况可以将 $n^2 \to n\log n$

适用于子结构相同且独立，适合并行计算的问题，可以配合动态规划进行优化

Basic Kit：中位数算法

1. 快速选择：平均 $O(n)$，最坏 $O(n^2)$

```
PARTITION(A,p,r)
  pivot := A[r];
  i := p-1;
  for j := p to r-1 do
    if A[j] < pivot then
      i := i+1;
      SWAP(A[i], A[j]);
  SWAP(A[i+1], A[r]);
  return i+1;

QUICK-SORT(A,p,r)
if p<r then
  q := PARTITION(A,p,r);
  QUICK-SORT(A,p,q-1);
  QUICK-SORT(A,q+1,r);

QUICK-SELECT(A,k,n)
  q := PARTITION(A,1,n);
  if q=k return A[q];
  else if q<k do
    return QUICK-SELECT(A-q,k-q,n-q);
  else do
    return QUICK-SELECT(A,k,q);
```

时间复杂度分析：

PARTITION 的时间复杂度：$n-1=O(n)$

快速排序：

平均：$A(n)=(n-1)+\sum_{i=0}^{n-1}\frac{1}{n}[A(i)+A(n-i-1)]$

注意到左右两部分的代价具有对称性

$\therefore A(n)=(n-1)+\sum_{i=0}^{n-1}\frac{2}{n}A(i)$

错位相减：$nA(n)=n(n-1)+2\sum_{i=0}^{n-1}A(i)$

$(n+1)A(n+1)=n(n+1)+2\sum_{i=0}^{n}A(i)$

$(n+1)A(n+1)=(n+2)A(n)+2n$

$\frac{A(n+1)}{n+2}=\frac{A(n)}{n+1}+\frac{2n}{(n+1)(n+2)}$

记 $B(n)=\frac{A(n)}{n+1}$　$\therefore B(n+1)-B(n)=\frac{2n}{(n+1)(n+2)}$

$\therefore B(n)=\sum_{i=1}^{n}\frac{2(i-1)}{i(i+1)}$（观察数量级近似 $\sum\frac{1}{i}=\log n$）

$=2\sum_{i=1}^{n}\frac{1}{i}-\frac{2}{i(i+1)}$

$=2\log n-4\sum_{i=1}^{n-1}-\frac{1}{i+1}$

$=2\log n+\frac{4}{n+1}-4=O(\log n)$

最坏　$A(n)=(n-1)+A(n-1)$　$A(1)=0$

得 $A(n)=\sum_{i=1}^{n-1}i=\frac{1}{2}n(n-1)=O(n^2)$

快速选择：

平均：由算法过程易知，当 $i<k$ 时，$T(i)=0$

$T(n)=(n-1)+\sum_{i=0}^{n-1}\frac{1}{n}T(i)$

$\therefore T(n)=(n-1)+\sum_{i=k}^{n-1}\frac{1}{n}T(i)$

$(n-1)T(n-1)=(n-1)(n-2)+\sum_{i=k}^{n-2}T(i)$

相减得 $nT(n)-(n-1)T(n-1)=2(n-1)+T(n-1)$

化简得 $T(n)=2-\frac{1}{n}+T(n-1)<2+T(n-1)$

易知 $T(n)=O(n)$

快速选择算法的另一种求法：引入随机变量，计算子问题规模期望

记 $m(n)$ 为子问题规模大小，则 $T(n)=T(m(n))+(n-1)$

$E(m(n))=\frac{1}{n}\sum_{i=0}^{n-1}\max(i,n-1-i)\le\frac{2}{n}\sum_{i=\lceil n/2\rceil}^{n-1}i\le\frac{3}{4}n$

$\therefore T(n)=T(\frac{3}{4}n)+(n-1)$　$a=1$　$b=\frac{4}{3}$　$f(n)=n-1$　$ab^p=1$　$\therefore p=0$

由 Akra-Bazzi 定理知，$T(n)=\Theta(n^0(1+\int_1^n\frac{u-1}{u}du))=\Theta(n-\log n)=\Theta(n)$

2. BFPRT 算法：通过严格确定枢轴，使最坏情况的时间复杂度也为 $O(n)$，设其为 k 划分的

每次划分都将根据枢轴将元素分为 4 个区域 ABCD，其中有 1 个区域将会被必然舍去

去掉的区域的规模大小至少为 $k*(\frac{1}{2}\times\lceil\frac{k}{2}\rceil-2)\ge\frac{k+1}{4}n-(k+1)$，则最多剩下 $\frac{3k+4}{4k}n+(k+1)$ 规模的问题

其中 $\lceil\frac{k}{2}\rceil$ 表示组数，$\frac{1}{2}$ 表示去掉一半，-2 表示保留中位数本组和不满的组，$k*$ 表示去掉每组 $\lceil\frac{k}{2}\rceil$，也就是一半元素

$T(n)\le T(\lceil\frac{n}{k}\rceil)+T(\frac{3k+4}{4k}n+(k+1))+O(n)$

假设 $\exists n_0>0,c>0$ 使 $\forall n\ge n_0$，$T(n)\le cn$ 成立　代入原式得：

$T(n)\le c\lceil\frac{n}{k}\rceil+c(\frac{3k+4}{4k}n+k+1)+O(n)\le cn-((\frac{1}{4}-\frac{3}{4k})cn-c_2n-(k+1)c)$

得当 $k\ge3$，$c>\frac{4kc_2n}{(k-3)n-4k(k+1)}$ 时成立　由证明过程可知，$k>3$

得证 $T(n)=O(n)$

3. 对顶堆：

概念不难理解，下面介绍堆操作的时间复杂度：

构建堆：高度为 h 的节点最多进行 h 次交换，第 k 层（高度为 $h-k$）节点个数最多为 $\frac{n}{2^{k+1}}$

总调整次数 $S=\sum_{k=0}^{h}\frac{n}{2^{k+1}}(h-k)$，取 $m=h-k$，得 $S=\sum_{m=0}^{h}\frac{mn}{2^{h-m+1}}=\frac{1}{2^{h+1}}\sum_{m=1}^{n}m2^m=\frac{1}{2^{h+1}}((n-1)2^{n+1}+2)$

$S=O(n)$ 因此构建堆的时间复杂度为 $O(n)$

调整堆：树的高度为 $\lfloor\log_2(n+1)\rfloor$，因此插入后调整复杂度为 $O(\log n)$

典例1：Strassen 矩阵乘法

普通矩阵乘法：复杂度 $\Omega(n^3)$，由3个 for 循环构成

分治矩阵乘法：划分为4个小矩阵，由8个 $\frac{n}{2}\times\frac{n}{2}$ 矩阵乘和 4个 $\frac{n}{2}\times\frac{n}{2}$ 矩阵加构成

$T(n)=8T(\frac{n}{2})+\Theta(n^2)$ 得 $T(n)=O(n^3)$

Strassen 乘法：通过减少矩阵乘使 $T(n)=7T(\frac{n}{2})+\Theta(n^2)$，得 $T(n)=O(n^{\log_2 7})=O(n^{2.81})$

$$\begin{bmatrix}A_{11} & A_{12}\\ A_{21} & A_{22}\end{bmatrix}\cdot\begin{bmatrix}B_{11} & B_{12}\\ B_{21} & B_{22}\end{bmatrix}=\begin{bmatrix}C_{11} & C_{12}\\ C_{21} & C_{22}\end{bmatrix}$$

$C_{11}=A_{11}B_{11}+A_{12}B_{21}$　　$C_{12}=A_{11}B_{12}+A_{12}B_{22}$

$C_{21}=A_{21}B_{11}+A_{22}B_{21}$　　$C_{22}=A_{21}B_{12}+A_{22}B_{22}$

现在构建如下的7个矩阵：

$P_1=A_{11}(B_{12}-B_{22})$

$P_2=(A_{11}+A_{12})B_{22}$

$P_3=(A_{21}+A_{22})B_{11}$

$P_4=A_{22}(B_{21}-B_{11})$

$P_5=(A_{11}+A_{22})(B_{11}+B_{22})$

$P_6=(A_{12}-A_{22})(B_{21}+B_{22})$

$P_7=(A_{11}-A_{21})(B_{11}+B_{12})$

则：

$C_{11}=P_5+P_4-P_2+P_6$

$C_{12}=P_1+P_2$

$C_{21}=P_3+P_4$

$C_{22}=P_5+P_1-P_3-P_7$

典例2. 汉诺塔问题

1. 标准汉诺塔：A B C

① $(n-1)A\to C$　② $1A\to B$　③ $(n-1)C\to B$

$T(n)=2T(n-1)+O(1)=2^{n-1}T(1)+2^{n-1}-1=2^n-1$

2. 四柱汉诺塔：A B C D

① $kA\to C$　② $(n-k-1)A\to D$　③ $1A\to B$　④ $(n-k-1)D\to B$　⑤ $kC\to B$

将前k个圆盘移动到某一中间柱，剩余的n-k个圆盘借助汉诺塔移动，最后将k个圆盘移回来

$T(n)=2T(k)+T(n-k)$　取$k=\sqrt{n}$时，近似得$T(n)=O(n^{\frac{3}{2}})$

3. 循环汉诺塔：A不能直接向B移动 或 $A\to C\to B\to A$ 必须依次移动

① $(n-1)A\to B$　② $1A\to C$　③ $(n-1)B\to A$　④ $1C\to B$　⑤ $(n-1)A\to B$

例 $n=2$：

$1A\to B$ ＜ $0A\to B$ / $1A\to C$ / $0B\to A$ / $1C\to B$ / $0A\to B$

$1A\to C$

$1B\to A$

$1C\to B$ ＜ $1A\to C$ / $1C\to B$

$1A\to B$

得$T(n)=3T(n-1)+2=3^{n-1}\times 2+3^{n-2}\times 2+\cdots+2=3^n-1$

4. 汉诺塔的迭代方法：

1. 间隔数：初始条件下，自上向下递增地标记圆盘序号

证明：第k号盘两次移动的间隔为2^k

首先根据 $nA\to B=(n-1)A\to C$、$1A\to B$、$(n-1)C\to A$ 的过程可知，k号盘将会被移动2^{n-k}次

且k号盘的移动是长为2^k-1不重叠空间的中点，空间之间有1的间隔，点间隔$=1+2^k-1=2^k$

平凡方法：

第m+1个盘移动前后，必须有m个盘子移动，已知规模为m的汉诺塔问题要移动2^m-1次

故移动第m+1个盘前后要移动$2^{m+1}-2$次，同时两个规模为m+1的汉诺塔问题之间还有一次第m+2盘的移动

故第m+1盘的移动间隔为$2^{m+1}-2+1+1=2^{m+1}$

二进制表示法：

根据汉诺塔问题的特点，可以认为盘的移动是二进制进位过程，

第m个盘子的移动必须依赖 m-1 个盘子在辅助盘上，即第m位进1必须使 m-1 位全为1

此时 S、A、D 身份交换，相当于所有低位连续1置0

汉诺塔行为的展开形式为：1. 将低位以自增形式置1

2. 自增1

3. 将低位置0

2. 移动方向：

第k号盘子 = 当k为奇数时，按 A→B→C→A 方向移动

当k为偶数时，按 A→C→B→A 方向移动

证明：当k=1时，A→B 成立，当k=3时，(1) A→B (2) A→C (3) B→C (4) A→B (5) C→A (6) C→B (7) A→B

假设对于 $\forall k \le m$，都满足以上移动方向规律，当k=m+1时

规定函数 H(k, S, A, D)，k是序号，S是源，A是辅助，D是目的

则 H(m+1, S, A, D) = H(m, S, D, A)，H(1, S, A, D)，H(m, A, S, D)

得：k的移动为 S→D ⇔ k-1的移动为 S→A→D

1. 当m是偶数时，其序列为 A→C→B→A

则对 A→C→B，m+1的移动为 A→B，对 B→A→C，m+1的移动为 B→C

对 C→B→A，m+1的移动为 C→A，得 m+1(奇)的移动为 A→B→C→A

2. 当m是奇数时，其序列为 A→B→C→A

则对 A→B→C，m+1的移动为 A→C，对 C→A→B，m+1的移动为 C→B

对 B→C→A，m+1的移动为 B→A，得 m+1(偶)的移动为 A→C→B→A

因此 k=m+1 时也成立，归纳得证

典例3，Monge阵列

4-6 (Monge 阵列) 对一个 $m\times n$ 的实数阵列 A，若对所有满足 $1\leqslant i<k\leqslant m$ 和 $1\leqslant j<l\leqslant n$ 的 i，j，k 和 l 有

$$A[i,j]+A[k,l]\leqslant A[i,l]+A[k,j]$$

则称 A 是 **Monge 阵列**(Monge array)。换句话说，无论何时选出 Monge 阵列的两行和两列，对于交叉点上的 4 个元素，左上和右下两个元素之和总是小于等于左下和右上元素之和。例如，下面就是一个 Monge 阵列：

10	17	13	28	23
17	22	16	29	23
24	28	22	34	24
11	13	6	17	7
45	44	32	37	23
36	33	19	21	6
75	66	51	53	34

a. 证明：一个数组是 Monge 阵列当且仅当对所有 $i=1,2,\cdots,m-1$ 和 $j=1,2,\cdots,n-1$，有

$$A[i,j]+A[i+1,j+1]\leqslant A[i,j+1]+A[i+1,j]$$

(提示：对于“当”的部分，分别对行和列使用归纳法。)

b. 下面数组不是 Monge 阵列。改变一个元素使其变成 Monge 阵列。(提示：利用(a)的结果。)

37	23	22	32
21	6	7	10
53	34	30	31
32	13	9	6
43	21	15	8

c. 令 $f(i)$ 表示第 i 行的最左最小元素的列下标。证明：对任意 $m\times n$ 的 Monge 阵列，$f(1)\leqslant f(2)\leqslant\cdots\leqslant f(m)$。

d. 下面是一个计算 $m\times n$ 的 Monge 阵列 A 每一行最左最小元素的分治算法的描述：

提取 A 的偶数行构造其子矩阵 A'。递归地确定 A' 每行的最左最小元素。

然后计算 A 的奇数行的最左最小元素。

解释如何在 $O(m+n)$ 时间内计算 A 的奇数行的最左最小元素(在偶数行的最左最小元素已知的情况下)。

e. 给出(d)中描述的算法的运行时间的递归式。证明其解为 $O(m+n\log m)$。

a. 充分性：取 $k=i+1$、$l=j+1$、代入易证

必要性：

①证明 Monge 矩阵的行单调性：

证明：Monge $\Leftrightarrow A_{ij}-A_{il}\leq A_{kj}-A_{kl}$ $(\forall i\leq k, j\leq l)$

②证明 Monge 矩阵的列单调性：

证明：Monge $\Leftrightarrow A_{kl}-A_{il}\leq A_{kj}-A_{ij}$ $(\forall i\leq k, j\leq l)$

以上两种性质均由原式直接变形可得

因此，若 A 满足 $A_{ij}+A_{(i+1)(j+1)}\leq A_{i(j+1)}+A_{(i+1)j}$

则满足 ① $A_{(i+1)(j+1)}-A_{(i+1)j}\leq A_{i(j+1)}-A_{ij}$

② $A_{(i+1)(j+1)}-A_{i(j+1)}\leq A_{(i+1)j}-A_{ij}$

因此，对 $\forall 1<i\leq m$，$1<j\leq n$

$A_{ij}-A_{i(j-1)}\leq A_{pj}-A_{p(j-1)}$，$p<i$

$A_{ij}-A_{(i-1)j}\leq A_{iq}-A_{(i-1)q}$，$q<j$

则对于 $\forall 1\leq i<k\leq m$，$1\leq j<l\leq n$

$A_{il}+A_{kj}-A_{ij}-A_{kl}$

$=A_{il}-A_{i(l-1)}+A_{i(l-1)}-A_{i(l-2)}+\cdots+A_{i(j+1)}-A_{ij}$

$-(A_{kl}-A_{k(l-1)}+A_{k(l-1)}-A_{k(l-2)}+\cdots+A_{k(j+1)}-A_{kj})$

对于每4项：$(A_{il}-A_{i(l-1)})-(A_{kl}-A_{k(l-1)})\leq 0$

故 $A_{il}+A_{kj}\leq A_{ij}+A_{kl}$

由此得出新的 Monge 矩阵判定方法：列差法

得到列差矩阵 $D_{ij}=A_{ij}-A_{(i+1)j}$

若对每一行都有 $D_{i(j+1)}-D_{ij}\geq 0$(单调不减)则为 Monge 矩阵

$D_{(i+1)j}-D_{ij}=A_{(i+1)(j+1)}-A_{(i+1)j}-A_{ij}+A_{i(j+1)}\geq 0$

c. 假设存在 $1\leq a<b\leq m$，使 $f(a)>f(b)$，则：

记 $f(a)=k$，$f(b)=l$，$A_{ak}=\min\{A_{aj}\mid 1\leq j\leq n\}$

$A_{bl}=\min\{A_{bj}\mid 1\leq j\leq n\}$，且 $k>l$

则 $A_{ak}+A_{bl}<A_{al}+A_{bk}$

∵ $a<b$ 且 $l<k$，故与 Monge 矩阵性质矛盾，

故 $\forall 1\leq a<b\leq m$，均有 $f(a)\leq f(b)$

d. 计算A的奇数行的最左最小元素：

记当前行数为 k，k 为奇数，$k+1$，$k-1$ 均为偶数，且均已知其最左最小元素 $f(k+1)$、$f(k-1)$

则 $f(k-1)\leq f(k)\leq f(k+1)$，记 $f(k)-f(k-1)=p_k$

则 $\sum_{i=1}^{m}p_i=n$，计算奇数行元素时间复杂度：

$\sum_{i=1}^{\frac{m}{2}}(1+p_i)=m+n=O(m+n)$

故 $T(m,n)=T(\frac{m}{2},n)+O(m+n)=O(n\log m+m)$

非构造解方法

对手论证 = 从系统视角下的算法

结合信息论、博弈论和决策树讨论算法时间复杂度上下界的手法

过程 = ① 明确不同情况下的对手操作 ② 证明对手无法区分输入实例 ③ 推导下界

例1 = 证明基于比较的排序 $T(n) = \Omega(n\log n)$

1. 对手论证 = 规定对于某元素未比较的元素状态为 N，比较后的较大元素为 W，较小为 L

确定数组中某元素的位置，需要 n 的信息量

比较前状态	对手应对	比较后状态	获得信息量
N、N	$a>b$	W、L	2
W、N	$a>b$	W、L	1
L、N	$a>b$	W、L	1
其它	—	—	0

可知每次比较最多可以获得 2 点信息量，因此每次构造 N,N 型比较，进行 $\frac{n}{2}$ 次

最后将得到 W、L 两个数组，这两个数组中的元素之间没有相互比较，均为 N，分别递归比较之

则可以构造 $T(n) = 2T(\frac{n}{2}) + \frac{n}{2}$，得 $T(n) = \Theta(n\log n)$，因此有 $T(n) = \Omega(n\log n)$

2. 决策树视角 = 设决策树的树高为 h，叶子个数为 n，则有 $n \le 2^h$

叶节点的个数对应一种排序结果，排序结果为输入元素的某一序列，有 k! 种，故 $n \ge k!$

则 $2^h \ge k!$ $h \ge \log_2(k!) \ge k\log k$ 也即 $T(n) = \Omega(n\log n)$

决策树分析比较问题具有明显优势

2*. 决策树分析平均时间复杂度

定义 = 外部路径 EPL = 根节点到所有节点的路径长度之和（决策树视角下的总代价）

$EPL(leaf) = 0$ $EPL(T) = EPL(T_L) + N_L + EPL(T_R) + N_R$ （N 表示树的叶节点个数）

决策树视角下的平均代价即为 $EPL(T)/L$

性质 = 越平衡的 2-tree 具有越小的 EPL

$EPL(T) \ge EPL(T_{平衡}) = L\lceil\log_2 L\rceil$，因此平均代价为 $\lceil\log_2 L\rceil$

本例中 $L = n!$，因此平均时间复杂度为 $O(n\log n)$

例2，同时选择最大和最小元素

这一类问题并不能应用决策树，因为算法中各个元素不平等，因此无法构建决策树从正反两方面证

对手论证 = $T(n) \geq \lceil \frac{3n}{2} \rceil - 2$

设每个元素有4种状态，N、W、L、WL，最大元素只会W，最小元素只会L

比较前状态	对手应对	比较后状态	获得信息量
N、N	a>b	W、L	2
W、N 和 WL、N	a>b	W、L或WL、L	1
N、L	a>b	W、L	1
W、W 和 L、L	a>b	W、WL和W、L	1
其它	-	-	0

得到最大最小值最少要 $2(n-1)$ 信息量

最多可以进行 $\lfloor \frac{n}{2} \rfloor$ 次2信息量的比较，以后要做 $2(n-1)-2\lfloor \frac{n}{2} \rfloor$ 次1信息量的比较

总比较次数 $= \lfloor \frac{n}{2} \rfloor + 2(n-1) - 2\lfloor \frac{n}{2} \rfloor = n + \lceil \frac{n}{2} \rceil - 2 = \lceil \frac{3n}{2} \rceil - 2$ 次比较

例3 = 选择第k小元素（可以认为是最快的取中位数构造算法）

第k大元素代表一种偏序关系，即其比k-1个元素要大，比n-k个元素要小

设每个元素初始有1枚金币，两两比较后赢的人将取得所有金币

因此两两比较后一定会有一个人没有金币，一个人有全部金币

一旦一个人的金币数大于k，则将其除去，并以放回其比较得到的金币

为了尽可能放慢"除去一个人"的速度，我们要使所有人金币尽可能接近k，可以等于k

此时有 $\lfloor \frac{n}{k} \rfloor$ 个人有 k 个金币，一个人有 $n \bmod k$ 个金币，$n - \lceil \frac{n}{k} \rceil$ 人没有金币（比较 $3n - \lceil \frac{3n}{k} \rceil$ 次）

这几个人之间相互比较，较大的一定会大于k，除去，最后会有 $\frac{n}{k}$ 个人被除去，也就是会有 $\lfloor \frac{n}{k} \rfloor$ 个人重新有1金币

有 $\lfloor \frac{n}{k} \rfloor$ 个人有 k-1 个金币，则这 $\lfloor \frac{n}{k} \rfloor + 1$ 个人再相互比较，每轮比较 $\lfloor \frac{n}{k} \rfloor$ 次，除去 $\lfloor \frac{n}{2k} \rfloor$ 个人

一共将比较 $\lfloor \frac{n}{2} \rfloor$ 轮，此时共除去 $\lceil \frac{n}{2} \rceil \lfloor \frac{n}{k} \rfloor$ 个人，余下 $\lfloor \frac{n}{2k} \rfloor$ 个人有 $2k$ 个金币，$n - \lceil \frac{n}{2} \rceil \lfloor \frac{n}{k} \rfloor - \lceil \frac{n}{2k} \rceil$ 个人无金币

重新构建k硬币组，进行 $\lceil \frac{n}{k}/2 \rceil$ 次比较，得 $\lfloor \frac{n}{k}/2 \rfloor$ 个人有 $2k$ 个金币，$n - \lceil \frac{n}{2} \rceil \lfloor \frac{n}{k} \rfloor - \lceil \frac{n}{2k} \rceil \lfloor \frac{n}{k}/2 \rfloor$ 个人无金币

为了方便计算，假设 $n \bmod k = 0$，$k \bmod 2 = 0$，得共进行 $\frac{3n}{2} + \frac{n}{2k}$ 次比较除去 $\frac{3n}{4}$ 个人

$A(n) = A(\frac{n}{2}) + \frac{n}{2} + \frac{n}{2k}$，$n \leq \min(k, n-k)$ 时 $A(n) \leq n-1$，$T(n) = A(n) + n - \lceil \frac{n}{2} \rceil$

展开得 $T(n) \leq n(1 - \frac{1}{2^{m-1}}) + \frac{1}{2^m} - 1 + n - \lceil \frac{n}{2} \rceil$，$m = \lceil \log_2 \frac{n}{k} \rceil \leq 2n - \frac{n}{k} - k - 1 + \frac{n}{k}$

取 $k = an$，$a \leq \frac{1}{2}$，则 $T(n) \leq (2-a)n - \frac{1}{a} + (a-1) < 2n$

4-5 （芯片检测） Diogenes教授有 n 片可能完全一样的集成电路芯片，原理上可以用来相互检测。教授的测试夹具同时只能容纳两块芯片。当夹具装载上时，每块芯片都检测另一块，并报告它是好是坏。一块好的芯片总能准确报告另一块芯片的好坏，但教授不能信任坏芯片报告的结果。因此，4种可能的测试结果如下：

芯片 A 的结果	芯片 B 的结果	结　论
B 是好的	A 是好的	两片都是好的，或都是坏的
B 是好的	A 是坏的	至少一块是坏的
B 是坏的	A 是好的	至少一块是坏的
B 是坏的	A 是坏的	至少一块是坏的

a. 证明：如果超过 $n/2$ 块芯片是坏的，使用任何基于这种逐对检测操作的策略，教授都不能确定哪些芯片是好的。假定坏芯片可以合谋欺骗教授。

b. 考虑从 n 块芯片中寻找一块好芯片的问题，假定超过 $n/2$ 块芯片是好的。证明：进行 $\lfloor n/2 \rfloor$ 次逐对检测足以将问题规模减半。

c. 假定超过 $n/2$ 块芯片是好的，证明：可以用 $\Theta(n)$ 次逐对检测找出好的芯片。给出描述检测次数的递归式，并求解它。

a. 假设有超过 $n/2$ 的芯片是坏的，则芯片两两比较的过程中，一定会出现两个坏芯片相互比较的可能。比较的结果可能是以上四种情况的任意一种。

由对手论证过程得，每种情况均要舍弃芯片，否则无法减小问题规模，算法无法终止

对于情况2、3、4：由对手论证得，只有舍弃全部芯片才能保证舍弃的坏芯片≥好芯片

且为了更多保留坏芯片，对手会使每次都舍弃一个好芯片和一个坏芯片，仍然保持坏芯片较多

对于情况1：已知两个坏芯片相互比较的情况一定多于两个好芯片相互比较的情况，则：

①若每次只舍弃1个芯片，则坏芯片依然比好芯片多

②若每次舍弃2个芯片，则坏芯片会选择其他情况，只会减少好芯片数量，坏芯片仍比好芯片多

综上，通过比较最后一定会得到坏芯片，坏芯片无法判断其他芯片的好坏，因此无法得到好芯片

b. 假设超过 $n/2$ 的芯片是好的，则：

对于情况2、3、4，将两个芯片均丢弃；对于情况1，丢弃1个芯片。仍能保持好芯片 $> \frac{n'}{2}$ ($n' = \frac{n}{2}$)

则将所有 n 个芯片两两比较（$\Theta(\frac{n}{2})$ 复杂度）后，问题的规模减小一半

c. $T(n) = T(\frac{1}{2}n) + \Theta(\frac{n}{2})$，由主定理可得 $T(n) = \Theta(n)$

优先权队列=

1. 基于无序数组/链表=

INSERT = O(1) 直接插入

EXTRACT-MIN = O(n) 遍历后取出最值

DECRASE-KEY = O(1)

1.2 基于有序数组或链表

O(n) 遍历插入调整

O(1) 取出端点

2. 基于堆=

INSERT = O(logn) 放入最后，再向上调整

EXTRACT-MIN = O(logn) 交换最后，再向下调整

DECRASE-KEY = O(logn)

BUILD = O(n) 建堆

3. 基于Fibnacci堆= 由多个树组成的松散结构，支持很快联合并和延迟剪切

数据结构= 使用环形链表维护子女，degree 记录子女数量，mark 位标记是否失去过节点，

Fib堆由若干最小堆有序树组成，树的根也由双向链表维护，使用min维护最小权值节点

同时使用degree-map[]来维护度数-节点映射关系，保证每个子树度数均不同

INSERT = O(1) 直接在Fib堆的子女加入一个叶子节点，更新min

UNION = O(1) 将两个子树(堆)的根节点合并入同层双向链表

EXTRACT-MIN = O(logn) 提取min节点，将其所有的子节点都上提至根节点同层循环链表，重新选择min节点，

(向上提的过程要跳过"失去过子节点"的节点，保证子树大小 $\geq F_{i+2}$

对合并同度数根节点= 遍历根双向链表，对于每一个子树x，若degree-map[x.d]=y，y≠x，则比较x和y的最小值，将最小值较大的树作为另一树的子树。重复直到双向性质恢复

时间复杂度 O(D(n)+logn)，D(n)是最大度的度数 = O(logn)

证明= 节点数下界= 度数为k的节点，必然至少包含 F_{k+2} 个子节点，F_i 是第i项的斐波那契数，$F_0=0, F_1=1$

证明= i=0，i=1时显然成立，假设对于所有i<k均成立，记x的度数为k，有k个子节点 y_i，y_i 的度至少为 i-1

故一共有 $\sum_{i=1}^{k} F_{i-1+2} + 1 = \sum_{i=1}^{k+1} F_i = F_{k+2}$ 因此当x=k时也满足条件

设Fib堆中一共有n个节点，最大度数为D(n)，则 $F_{D(n)+2} \leq n$，则 $D(n)+2 \leq \log_{\phi} n = O(\log n)$

哈希表

1、链接法闭散列 = (假设使用简单均匀散列)

不成功查找的平均时间 = $\Theta(1+\alpha)$, $\alpha = \frac{n}{m}$

不成功 = 查找到链表的末尾, 链表长度的期望为 α

成功查找的平均时间 = $\Theta(1+\alpha)$, $\alpha = \frac{n}{m}$

设示性变量 X_{ij} 表示 $A[i]$ 和 $A[j]$ 被放入同一个槽中

$E[\text{探查个数}] = E[\frac{1}{n}\sum_{i=1}^{n}(1+\sum_{j=i+1}^{n}X_{ij})]$ 因为每次都在链表头插入, 因此后插入的会先被探查, 为从计开始

$= \frac{1}{n}\sum_{i=1}^{n}(1+\sum_{j=i+1}^{n}E(X_{ij})) = \frac{1}{n}\sum_{i=1}^{n}(1+\sum_{j=i+1}^{n}\frac{1}{m}) = \frac{1}{n}(n+\frac{n(n-1)}{m}) = 1+\frac{n-1}{2m}$

$E[T(n,m)] = 1 + E(\text{探查个数}) = 2+\frac{n-1}{2m} = \Theta(1+\alpha)$

2、一次探查/二次探查开散列

因为每次探查序列只和初始位置有关, 所以探查序列一共有 m 种

不成功查找的平均时间 = $\Theta(\frac{1}{1-\alpha})$, $\alpha = \frac{n}{m}$ (插入相同)

定义事件 A_i 表示第 i 次探查到被占用的槽, X 表示不成功的探查次数

则 $P\{X \geq i\} = P\{A_1 \cap A_2 \cap \cdots \cap A_{i-1}\} = P\{A_1\} \cdot P\{A_2|A_1\} \cdots P\{A_{i-1}|A_1 \cap A_2 \cdots\}$

$= \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \frac{n-i+2}{m-i+2} \leq (\frac{n}{m})^{i-1} = \alpha^{i-1}$

$E(X) = \sum_{i=1}^{\infty}P(X \geq i) \leq \sum_{i=1}^{\infty}\alpha^{i-1} = \frac{1}{1-\alpha}$

成功探查的平均时间 = $O(\frac{1}{\alpha}\ln\frac{1}{1-\alpha})$, $\alpha = \frac{n}{m}$

k 的探查序列 = k 的插入探查序列 = k 的不成功探查序列

设 k 是第 i 个被插入表中的关键字, 则期望探查次数为 $\frac{1}{1-\frac{i-1}{m}} = \frac{m}{m-i}$

$E(\text{探查时间}) = \frac{1}{n}\sum_{i=0}^{n-1}\frac{m}{m-i} = \frac{m}{n}\sum_{k=m-n+1}^{m}\frac{1}{k} \leq \alpha\int_{m-n}^{m}\frac{1}{x}dx = \frac{1}{\alpha}\ln\frac{1}{1-\alpha}$

红黑树

定义：染色、2-tree结构，二叉搜索树，

黑色深度：从根到达该节点路径上黑色节点的个数。(不包括根节点，根节点的黑色深度为0)

红黑树的直接定义：

根为黑色，所有外部节点为黑色。(根为红色则为ARB树)

红色节点不能连续出现，即没有任何一个红色父节点有红色子节点。

以任意节点为根的子树中，所有的外部节点黑色深度相等。(=该节点黑高度=该子树黑色高度)

红黑树递归定义：

如果根节点=外部节点，构成RB_0

h>1的ARB_h，则左右子树均为RB_h

h>1的RB_h，则左右子树均为RB_{h-1}或ARB_h

假设T为RB_h，则：

① 内部黑色节点 $\geq 2^h-1$

② 内部节点 $\leq 4^h-1$

③ 黑色节点的普通高度至多是其黑色高度2倍

假设T为ARB_h，则：

① 内部黑色节点 $\geq 2^h-2$

② 内部节点 $\leq \frac{1}{2}\times 4^h-1$

③ 黑色节点的普通高度至多是其黑色高度2倍

6.4. 记T为一棵ARB或RB，其内部黑色节点个数为 $B(T)$，内部节点个数为 $N(T)$，普通高度 $H_S(T)$，黑高度 $H_B(T)$

对于ARB_h，其根节点为红色，左右子树均为RB_{h-1}

则 $B(ARB_h)=2B(RB_{h-1})$　　$N(ARB_h)=1+2N(RB_{h-1})$

$H_S(ARB_h)=H_S(RB_{h-1})+1$　　$H_B(ARB_h)=H_B(RB_{h-1})$

对于RB_h，其根节点为黑色，左右子树为RB_{h-1}或ARB_h

则 $B(RB_h)=2B(RB_{h-1})+1=B(RB_{h-1})+B(ARB_h)+1=2B(ARB_h)+1$

$N(RB_h)=2N(RB_{h-1})+1=N(RB_{h-1})+N(ARB_h)+1=2N(ARB_h)+1$

$H_S(RB_h)=\max(H_S(RB_{h-1}),H_S(ARB_h))+1$

$H_B(RB_h)=\max(H_B(RB_{h-1}),H_B(ARB_h))+1$

∵ $H_S(ARB_h)=H_S(RB_{h-1})+1$　　$H_B(ARB_h)=H_B(RB_{h-1})$

∴ $H_S(RB_h)_{max}=H_S(RB_{h-1})_{max}+2$　　$H_B(RB_h)=H_B(RB_{h-1})+1$

∵ $H_S(RB_0)=0$　$H_B(RB_0)=0$　∴ $H_S(RB_h)_{max}=2h$　$H_B(RB_h)=h$

$H_S(RB_h)/H_B(RB_h)\leq H_S(RB_h)_{max}/H_B(RB_h)=2$

故引理6.1.3、RB_h任何节点的普通高度至多为黑色高度的2倍　得证

$H_S(ARB_h)_{max}=H_S(RB_{h-1})_{max}+1=2h-1$

$H_B(ARB_h)=H_B(RB_{h-1})=h-1$

ARB_h中黑色高度最大的黑色节点是根节点的子女节点，黑高度为$2h-2$

$H_S(ARB_h)/H_B(ARB_h)<H_S(ARB_h)_{max}/H_B(ARB_h)=2$

故引理6.2.3、ARB_h任何节点的普通高度至多为黑色高度的2倍　得证

$B(RB_h)=2B(RB_{h-1})+1$

$=B(RB_{h-1})+B(ARB_h)+1=3B(RB_{h-1})+1$

$=2B(ARB_h)+1=4B(RB_{h-1})+1$

∴ $B(RB_h)_{min}=2B(RB_{h-1})_{min}+1$　$B(RB_0)=0$

展开后可得 $B(RB_h)_{min}=2^h-1$

故引理6.1.1、RB_h有不少于2^h-1个内部黑色节点，得证

$B(ARB_h)_{min}=2B(RB_{h-1})_{min}=2^h-2$

故引理6.2.1、ARB_h有不少于2^h-2个内部黑色节点，得证

$N(RB_h)=2N(RB_{h-1})+1$

$=N(RB_{h-1})+N(ARB_h)+1=3N(RB_{h-1})+2$

$=2N(ARB_h)+1=4N(RB_{h-1})+3$

∴ $N(RB_h)_{max}=4N(RB_{h-1})_{max}+3$

∵ $N(RB_0)=0$　∴ 展开后得 $N(RB_h)_{max}=4^h-1$

故引理6.1.2、RB_h最多有4^h-1个内部节点，得证

$N(ARB_h)_{max}=2N(ARB_h)_{max}+1=\frac{1}{2}4^h-1$

故引理6.2.2、ARB_h最多有$\frac{1}{2}4^h-1$个内部节点，得证

红黑树的旋转：（解决红黑树平衡问题的方法）

1、左单　2、右单　3、先左后右　4、先右后左　（5、下放黑结点）

1、插入过程：

红黑树的每次插入都只会插入红色节点，因此树不空的情况下，只会违反"不能出现连续红节点"

当插入的节点z与z.p为连续红节点时，z.p.p一定是黑节点，考察z.p.p.c（叔节点）

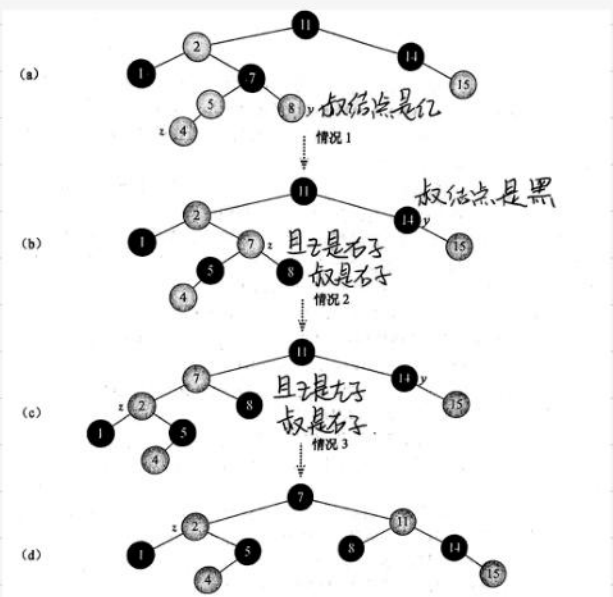
1、当叔节点是红节点时：

直接"下放黑节点"，祖父节点置红，父/叔节点置黑

2、当叔节点是黑节点时：

若叔节点和自己"异向"（例：叔节点是祖父节点右子，自己是父节点左子），单向旋转

若叔节点和自己"同向"，先单向旋转至异向，之后再反向旋转（双旋转）



2、删除过程：

首先规定替换方式：当删除z时，若只有一个子树，用其替换z，若其有两个子树，则根据二叉搜索树的删除原则，将z的最接近后继y来替换z，之后递归删除y。因为y一定没有左子树，故y的最接近后继有天然的配角，记为x（但只有一种特殊情况，也即y是黑，x也是黑 = $y_x^{\circ\circ}$）

替换时颜色也要替换，因此记录z.OriginalColor，y.OriginalColor，x.OriginalColor

①若y是红色的：则y替换z的过程对y所在的子树没有影响，直接替换染色即可

②若y是黑色的 & x是红色的：依然是直接移动且赋值

③若y是黑色的 & x是黑色的：则此时x（原y位置）是一个双黑色节点

现在我们设计一个算法的函数，使其可以恢复RBT性质

首先我们知道，x可能没有子女，但是一定会有父节点，因此要将额外的黑上移（假设x是左子）

①若x的兄弟w是红色的：则x.p是黑色的，将w和x.p做一次左旋即可

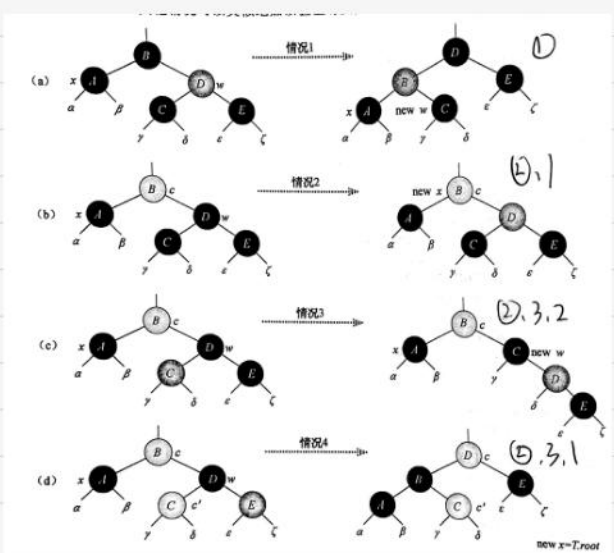
②若x的兄弟w是黑色的：

1. 若w的两个孩子都是黑色的：w侧无法调整，将x、w、黑色"减一"，x.p黑色"加一"，
2. 若w的两个孩子都是红色的：只要将w的黑色下放即可，之后即可左旋
3. 若w只有一个孩子v是红色的

3.1 v是右子：将w和x.p左旋，将w置红，v置黑，x黑色"减一"，x.p黑色"加一"

3.2 v是左子：将v和w右旋，之后进入3.2情况

若x.p原来也是黑色，则递归调用之，直至x没有父节点



并查集

操作 = FIND(ai) 返回ai所在的等价类代表元

UNION(ai, aj) 合并ai和aj的等价类

对于共n个元素的长为l的操作序列 = 最坏时间复杂度 O(nl)

普通并 + 普通查 = 构建极不平衡的并查树, O(nl)

加权并 + 普通查 = 构建平衡并查树, O(n + l logn)

加权并 + 路径压缩查 = 构建最矮并查树, $O((n+l)\log^* n) \approx O(n+l)$

d叉堆

A[1]是d叉堆的根节点

14.3 父节点 = $\lfloor \frac{i-2}{d} + 1 \rfloor$

式中 i-2表示去掉自己和根节点以外的节点数

⌊(i-2)/d⌋表示这些节点里有多少子节点序列完整的节点, 相当于层次序

遍历至父节点(跳过根节点)的节点个数, 也就是父节点下标减1

最后+1补足根节点, 得到父节点的下标

子节点 = d(i-1)+j+1

式中i-1表示去掉自己的所有节点, 如果能排到i的子节点,

那么1…i-1这些节点的子节点也应该排满,

因此乘d之后得到了所有已经排好的节点个数, 加1补上根节点

d(i-1)+1之后的d个节点就是i节点的d个子节点.

锦标赛

19.5 习题 4.4 给出了一个将 5 个元素进行排序的最优算法，记为 SORT-FIVE。下面考虑受限的 n 个元素选择问题，此时仅能使用 SORT-FIVE 算法来获得元素间的大小关系，对 SORT-FIVE 算法的一次调用记为 1 个关键操作。

1）如何使用 SORT-FIVE 算法找到 n 个元素中的最大值？请证明你的算法是最优的。

2）如何使用 SORT-FIVE 算法找到 n 个元素中的第二大值？请证明你的算法是最优的。

19.5 1) 将n个元素分成每组有5个元素的[n/5]组, 分别进行将5个元素进行排序的最优算法.

这样的操作将问题规模缩减成了n/5, 直到问题规模≤5时得到最大值

因此只要调用$\lfloor \log_5 n \rfloor$次, 每次调用5^{i-1}次算法.

总共调用$\sum_{i=1}^{\lfloor \log_5 n \rfloor} 5^{i-1} = \frac{n-1}{4}$次算法.

这个算法显然是最优的, 因为算法每次将排除4个元素, 若调用$\frac{n-1}{4}$次, 则有4个元素没有被排除, 则由对手论证可以将最大元素置入这些元素之中

2) 考虑一种"竞赛和低选者"模型, 假设最后一次比赛由5个人进行

则合计最后两场比赛, 共有25人,

A_1	B_1	C_1	D_1	E_1	最后一次比较
A_2	B_2	C_2	D_2	E_2	则第2大值可能出现在图中的椭圆区域内
A_3	B_3	C_3	D_3	E_3	右最后一次比较得到最大元素后, 还需将其倒
A_4	B_4	C_4	D_4	E_4	数第2次比较中第2大的元素与其他元素比较.
A_5	B_5	C_5	D_5	E_5	因此对比可以进行4次比较, 即$\frac{n}{5}+4$次
倒数第2次比较					显然是最优算法, 证毕. 问

图=

节点的颜色分类=

白色=还没有遍历到, TE

灰色=正在遍历,没有结束 BE

黑色=结束遍历 DE/CE

边的分类=

TE=(根据相应算法)构成遍历生成树的边 白色

BE=指向祖先结点的边 灰色

DE=指向后继结点的边 黑色(无向图没有)

CE=指向无明确偏序关系的点(其他子树上的点)的边 黑色(无向图没有)

有向无环图 ⇔ 存在拓扑排序

DFS性质=

深度优先 DFS(v)=

```
v.color := GRAY
(v的前序处理)
foreach neighbor w of v do
    if w.color = WHITE then
        (处理"有向边vw")
        DFS(w)
        (处理"有向边"wv")
    else
        (检查vw是什么类型的边)
(v的后序处理)
v.color := BLACK
```

性质1=活动区间,记录节点的发现时间和结束时间

1) w是v的后继节点 ⇔ active(w) ⊆ active(v)

2) w和v没有前缀后缀关系 ⇔ active(w) ∩ active(v) = ∅

3) vw是CE ⇔ active(w) 在active(v)前面

vw是DE ⇔ 当且仅当 active(w) ⊂ active(x) ⊂ active(v)

vw是TE ⇔ 不存在x使active(w) ⊂ active(x) ⊂ active(v) 且active(w) ⊂ active(v)

vw是BE ⇔ active(v) ⊂ active(w)

证明=1、TE

性质2=白色路径定理: 有向或无向图 $G=(V,E)$ 的深度优先森林中，结点 v 是结点 u 的后代当且仅当在 $u.s$ 存在一条从 u 到 v 的全部由白色结点构成的路径

证明: $\Rightarrow$: ①如果 $v=u$，则显然成立，在 $u.s$ 时尚未被染成白色

②如果 v 是 u 的真后代。则由性质1可知，$v.f<u.f$，$v.s>u.s$，则 v 在此时一定为白色

且由于 v 可以是 u 的任意后代，因此从 u 到 v 的唯一简单路径上的所有点都是白色

$\Leftarrow$: 假设 $u.s$ 时存在一条从 u 到 v 的白色路径，但 v 不是 u 在DFS树的后代。

不失一般性，假设从 u 到 v 路径上除结点 v 以外的每个结点都是 u 的后代。设 w 为路径上点 v 的前驱

根据性质1，有 $w.f\le u.f$，故 $u.s<v.s<v.f\le w.f\le u.f$，则由性质1，$v$ 是 u 的后继(后代)

故假设不成立，则有若存在一条从 u 到 v 的白色路径，则 v 一定是 u 在DFS树的后代

性质3=无向图DFS不会有DE和CE(边的类型↔点的颜色定义嵌套，因此不能直接假设点的颜色，而是要根据过程推断颜色)

证明=设 (u,v) 是 G 的任意一条边，假设 $u.s<v.s$，则当 u 是灰色时将DFS遍历到 v，在第一次遍历到 u 或 v 时=

①若第一次探索边 $u-v$ 时是 $u\to v$ 方向。则此时 v 一定是白色，故 $u\to v$ 是TE

②若第一次探索边 $u-v$ 时是 $v\to u$ 方向。则此时 $v\to u$ 指向灰色节点，故 $u\to v$ 是BE

SCC算法=使用DFS在有向图 $G=(V,E)$ 中寻找强连通分量

性质4=设 C 和 C' 为有向图 $G=(V,E)$ 的两个不同的强连通分量，假如存在一条边 $u\to v\in E$，$u\in C$，$v\in C'$，则 $f(C)>f(C')$，在 G^T 中，$f(C^T)<f(C'^T)$

证明=①若 $s(C)<s(C')$，设 x 为 C 中最早被发现的结点，则在 $x.c$，C 和 C' 中的所有节点都是白色的。

则由白色路径定理，C 和 C' 中的所有节点都是 x 的后代，则 $f(C')<f(C)=x.f$

②若 $s(C)>s(C')$，设 y 为 C' 中最早被发现的结点。因为 C 和 C' 是不同的强连通分量且 $u\to v\in E$，故对 $\forall a\in C'$，$b\in C$ 不存在边 $a\to b\in E$，则此时 C 和 C' 中的所有节点均为白色，C' 中的所有节点都是 y 的后代。则 $f(C')=y.f$ 时，C' 中所有节点均为黑色节点，C 中所有节点均为白色节点，$f(C)>d(C)>f(C')>d(C')$

Kosaraju=1.使用DFS(G)计算各点的finishtime，放入队列

2.将 G 转置得到 G^T

3.按照finishtime从大到小的顺序，依次对 G^T 中未访问的节点DFS，每次DFS得到的顶点构成一个SCC

正确性证明=假设步骤3中生成的前 k 棵树都是SCC。当 $k=0$ 时假设显然成立。

对第 $k+1$ 棵树，设其根结点为 u，处在SCC C 中。则 $u.f=f(C)>f(C')$，C' 是未被访问过的强连通分量。

则 $f(C')<f(C)$，$f(C'^T)>f(C^T)$，G^T 只存在 C' 向 C 的边，同时对于 $\forall$ 已访问过的SSC C''，只存在 C 向 C'' 的边

而 C'' 中的点已被访问，不构成DFS树，故 u 点出发的DFS树得到 C 强连通分量

拓展：第一次遍历不能使用BFS，第二次可以

2. 在第2次遍历过程时，当一个能节点从队列中dequene出来时一定是某节点
在入队的时候是按照相对于起始点的距离远的从小到大先后入队。故各个强连通的首节点也会按照原来的顺序出队。同时在dfs回时遍历得到的点集的交集一定是强连通片

如果不使用两个矩阵，算法是错误的。

（假设一个强连通片首节点的先出现队列，通过对首节点p的遍历，必然标记该强连通片的所有点。
假设在某一轮遍历时，o在dfs'有路径通往另一个强连通片S，S的首节点为b，则dfs'有在o→b路径C中存在b'o的路径，记起始点为O，δ(O,o)<δ(O,b)则o先放入队列，算法会错误）

因为不存在于f(C)与f(C')的关系，只存在δ(o,C)和δ(o,C')的关系，即这个关系与连通片性质无关

Tarjan 使用一个栈

index[u] = u.starttime

low[u] = min(index[u], low[u], low[v]) for each u→v and v is in stack

当DFS到u时：

标记u为已访问，index[u] = low[u] = time++, push(u)

遍历所有的邻接顶点v：

若v未访问，则DFS(v)

若v已访问且在栈中，则low[u] = min(low[u], index[v])（在栈中意味着没有被匹配给其它SCC）

检查是否是SCC根：若low[u] = index[u]，则从栈中弹出一直到u，这些顶点是SCC

正确性证明：low[u]表示u在DFS树中能通过BE回溯到祖先的最小值，若low[u] ≠ u.start则说明u的根的树中存在BE。DFS后序处理，index[u] = low[u]，则u是SCC的最早被发现的节点，则栈中从栈顶到u的元素构成SCC，因为：

定义⇔这些顶点能通过BE回到u，且这些顶点是u的DFS后代，对u可达⇔双向可达，是GCC且DFS中每个顶点都被遍历一次

Gabow：index[u] = u.starttime

使用两个栈

当DFS到u时：

标记u为已访问，index[u] = time++，压入元栈和路径栈

遍历所有的邻接顶点v：

若v未访问，DFS(v)

若v已访问且在路径栈中，保留（不做处理）

若路径栈顶有边指向u，则一直弹出至将u弹出，得到SSC

BFS性质=

性质1= 队列性质= 设BFS在图G上运行时，队列Q包含的结点为$(v_1, \dots, v_r)$，v_1是队列的头，v_r是队列的尾，则$v_r.d \le v_1.d+1$，且$v_i.d \le v_{i+1}.d$

证明= 初始情况下，Q中只包含源结点S，性质成立。下面证明在每次出队和入队的过程都成立=

将结点v加入队列时，该结点成为结点v_{r+1}，且删除了结点u。根据归纳过程，$v.d \ge u.d$，

故$v_{r+1}.d = v.d = u.d+1 \le v_1.d+1$，且$v_i.d \le u.d+1$，故性质成立。

性质2= BFS的正确性= 设G=(V,E)为一个有向/无向图，BFS以S为源结点在图G上运行，在算法执行过程中，BFS将发现所有S可达的结点v，并在算法终止时，$v.d = \delta(s,v)$。且S→V的一条最短路径为$\delta(s,v.\pi) + \delta(v.\pi, v)$，$v.\pi$表示在BFS中v的父节点。

证明= 假定某些结点获取的d值不等于最短路径距离，记作v，其最短路径距离为$\delta(s,v)$

$v.d \ne \delta(s,v)$，显然$v.d > \delta(s,v)$。设u为S到v最短路径上v的直接前驱，则$\delta(s,v) = \delta(s,u)+1$

故$v.d > \delta(s,v) = \delta(s,u)+1 = u.d+1$。当v从BFS对列中取出时，我们讨论v的颜色=

① 如果v是白色，则$v.d = u.d+1$，则矛盾

② 如果v是灰色，则存在某点w，使$v.d = w.d+1 \le u.d+1$，矛盾

③ 如果v是黑色，则$v.d \le u.d$，矛盾

综上，假设不成立。

```
BFS(G,s)
  for each vertex u ∈ G.V - {s}
      u.color = WHITE;  u.d = ∞;  u.π = NIL;
  s.color = GRAY;  s.d = 0;  s.π = NIL;
  Q = ∅.  ENQUEUE(Q,s);
  while Q ≠ ∅
    u = DEQUEUE(Q)
    for each v ∈ G.Adj[u]
      if v.color == WHITE
        v.color = GRAY;  v.d = u.d+1;  v.π = u;
        ENQUEUE(Q,v)
    u.color = BLACK
```

最小生成树=

1、证明BestFS贪心算法框架=

定理=设G=(V,E)是一个在边E上定义了实数值权重w的连通无向图，设集合A为E的一个子集，且A包括在图G的某棵最小生成树中，设(S, V-S)是图G中尊重集合A的任意一个切割，又设(u,v)是横跨切割(S, V-S)的一条轻量级边，那么边(u,v)对于集合A是安全的。

设T是一棵包括A的最小生成树，并假定T不包含轻量级边(u,v)，否则，我们已经证明完毕，现证明A∪{(u,v)}也被包含在另一棵最小生成树T'中，(u,v)对A是安全的

从u到v有另一条非(u,v)的路径p(否则最小生成树一定包含(u,v))，设p中边(x,y)是跨越(S, V-S)切割的某边，且这条边一定不在A中。若T不包含边(u,v)，则T一定包含(x,y)。

由假设可知，w(u,v) ≤ w(x,y)，故w(T') = w(T) − w(x,y) + w(u,v) ≤ w(T)

因T是最小生成树，故w(T') = w(T)，T'也是一棵最小生成树，(u,v)对A是安全的

由此我们证明了新加入一条权值最小的边，树扩张后还是最小生成树

进而得到了针对森林视角的kruskal算法和针对树视角的Prim算法

2、两种BFS算法=

1、kruskal算法=依次加入所有边中权值最小的边

使用最小堆+并查集的实现，时间复杂度O((V+E)logE)

2、Prim算法=依次加入与已构建的最小生成树相连的边中权值最小的边

使用最小堆记录点和点的权值，不断弹出点、更新图状态，时间复杂度O((V+E)logV)

扩展=针对稀疏图的最小生成树的Prim算法优化

方法1=预处理缩边法=

对每一个结点u，选择与结点u经接的边中最小权重的边(u,v)，将其加入到正在构建的最小生成树里，然后对所有选择的边进行收缩，收缩过程使用并查集实现，用一个点代表相连的所有点，进而重命名所有的边。多条边可能被命为相同的名，在这种情况下，重命名的边中只留下原始权重最小的边，认为这一过程的时间复杂度为O(E)，将使点的个数急剧减半。

执行缩边k次，$k=\log(\log V)$时，再次执行Prim算法，时间复杂度为O(E log(logV))

方法2=斐波那契优化

斐波那契堆的提取最小元素插入时间复杂度为O(logn)，插入元素/调整元素的时间复杂度为O(1)

每个顶点被插入和弹出一次，时间复杂度为O(VlogV)，每条边调用一次调整元素时间复杂度O(E)

一共时间复杂度为O(E+VlogV)

拓展：次优生成树

6-1 （次优最小生成树） 设 $G=(V, E)$为一连通无向图，其权重函数为 w：$E \to \mathbf{R}$，假定 $|E| \geqslant |V|$并且所有的权重都互不相同。我们定义一棵次优最小生成树如下：设$\mathcal{T}$为 G 的所有生成树的集合，T'为 G 的一棵最小生成树。那么**次优最小生成树**是生成树 T，其满足 $w(T)=\min_{T'' \in \mathcal{T}-\{T'\}} \{w(T'')\}$。

a. 证明：最小生成树是唯一的，但次优最小生成树则不一定是唯一的。

b. 设 T 为 G 的一棵最小生成树。证明：图 G 包含边 $(u, v) \in T$ 和边 $(x, y) \notin T$，使得 $T-\{(u, v)\} \cup \{(x, y)\}$是 G 的一棵次优最小生成树。

c. 设 T 为 G 的一棵最小生成树，对于任意两个结点 u，$v \in V$，设 $\max[u, v]$表示树 T 中从结点 u 到结点 v 的简单路径上最大权重的边，请给出一个 $O(V^2)$时间复杂度的算法来计算 $\max[u, v]$。

SMST（次优生成树）= 总权值第二大的生成树，与MST至少有一边不同

设计如下SMST生成算法：

1、首先得用BestFS算法得到最小生成树T'，权为W，记录新的树的总权值W'=∞

2、设新得到的SMST为T 使用数组Dir[v]记录各点相对于T中任意点a的直连距离，

使用Dir[v]表示对插中键为v（点v）的项的值

3、从T'中的某个叶子节点r开始遍历（基于BFS框架）

```
Push v in stack St while BFS(T');
r= St.pop(); v= St.top(); e=(r,v) in T';
while (r&v)
    Re = nullptr; Re'=nullptr; minw = INT_MAX;
    for each e'=(r,m) in G=
        if e' is not in T' and m is in St =
            Dir[m] = min{Dir[m], e'.w - e.w};
            if Dir[m] ≠ 0 and Dir[m] < minw =
                Re = e; Re'= e'; minw = Dir[m];
Replace Re with Re' in T' to get SMST;
```

单源最短路径：

1、单源最短路径：

2、单目的地最短路径：

3、所有结点对最短路径：

4、负权重的边：要求没有负权值的环路，Dijkstra要求边权非负，Bellman-Floyd可以侦测到负权环路

1、动态规划基石：最短路径的子路径也是最短路径，最短路径上不存在环路

2、"松弛"：记录两点之间的最短路径估计，用新的路径与之进行比较

路径松弛性质：如果$P=\langle v_0,\dots,v_k\rangle$是从源结点$s=v_0$到结点$v_k$的一条最短路径，并且我们对P中所有边的松弛顺序为$(v_0,v_1),(v_1,v_2),\dots,(v_{k-1},v_k)$，则$v_k.d=\delta(s,v_k)$

1. Bellman-Ford 算法

```
BELLMAN-FORD(G,w,s)
    INIT-SINGLE-SOURCE(G,s)
    for i=1 to |G.V|-1 :          循环|G.V|-1次，因为BFS树的边最多|G.V|-1条
        for each edge(u,v)∈G.E :  对边进行松弛操作，判断边是否在长为i的最短路径上
            RELAX(u,v,w)          时间复杂度O(VE)
    for each edge(u,v)∈G.E :      由路径松弛性质可以证明算法正确性
        if v.d > u.d + w(u,v) :   判断是否有负权回路
            return false;         没在在负权环路$P=\langle v_1,\dots,v_k\rangle$，$\sum_{i=1}^{k} w(v_{i-1},v_i)<0$
    return true;                  假设B-F算法返回True，则$v_i.d\le v_{i-1}.d+w(v_{i-1},v_i)$
                                  $\sum_{i=1}^{k} v_i.d \le \sum_{i=1}^{k}(v_{i-1}.d+w(v_{i-1},v_i))=\sum_{i=1}^{k} v_{i-1}.d+\sum_{i=1}^{k} w(v_{i-1},v_i)$
                                  $\because \sum_{i=1}^{k} v_i.d=\sum_{i=1}^{k} v_{i-1}.d$，故$0\le\sum_{i=1}^{k} w(v_{i-1},v_i)$与题矛盾
                                  故如果有负权回路就会返回FALSE
                                  （如果没有负权回路则通过三角不等式证明得到返回TRUE）
```

2. Dijkstra

```
DIJKSTRA(G,w,s)
    INIT-SINGLE-SOURCE(G,s)
    S=∅      Q=G.V
    while Q≠∅
        u=EXTRACT-MIN(Q)          不断地取出最近的点u
        S=S∪{u}
        for each v∈G.Adj[u]       根据u的所有边松弛
            RELAX(u,v,w)          时间复杂度和优先队列实现有关，O(Vk)
```

Dijkstra 正确性证明：证明 $u.d = \delta(s,u)$，即证明对于每一个 $v \in S$，$v.d = \delta(s,v)$

初始化：$S = \emptyset$，性质显然成立；$S - \{s\}$ 时性质也显然成立

保持：假设 $\exists u$，使 u 在被加入到 S 时，$u.d \neq \delta(s,u)$ 首次成立，且至少存在一条从 s 到 u 的路径。

设从 s 到 u 的最短路径为 p。在将 u 加入到集合 S 之前，路径 p 连接的是集合 S 中的一个结点 s 和 $V-S$ 的一个结点 u。设 p 上第一个满足 $y \in V-S$ 的结点 y，设 $x \in S$ 为结点 y 在 p 上的前驱，将 p 分解为 $s \xrightarrow{p_1} x \to y \xrightarrow{p_2} u$

下面证明 u 加入到集合 S 时，$y.d = \delta(s,y)$：

$x \in S$，且 u 是第一个不满足 $u.d \neq \delta(s,u)$ 的集合内节点，故 $x.d = \delta(s,x)$，此时边 (x,y) 将被松弛，

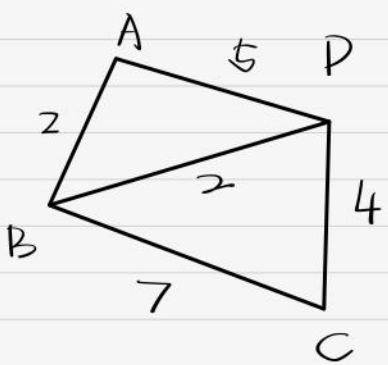
由“最短路径子路径也是最短路径”和路径松弛性质知 $y.d = \delta(s,y)$

$y.d = \delta(s,y) \leq \delta(s,u) \leq u.d$，而当 u 选择时，y 还没有被选择，因此 $u.d \leq y.d$

故 $y.d = \delta(s,y) = \delta(s,u) = u.d$　$\delta(s,u) = u.d$ 与 u 的性质矛盾。

故 u 在被加入到集合 S 时有 $u.d = \delta(s,u)$，并且该等式在随后的所有时间内保持成立

终止：$S = V$，故对所有节点 $u \in V$，$u.d = \delta(s,u)$



Dijkstra：

	A	B	C	D
	0	?	?	?
A	0	2	?	5
B	0	2	9	4
D	0	2	8	4
C	0	2	8	4

Bellman-Ford：

	A	B	C	D
0	0	?	?	?
1	0	2	?	5
2	0	2	9	4
3	0	2	8	4

x步可达的最短路径

贪心算法=

贪心算法框架=

1. 贪心选择性质=通过每一步的局部最优选择，能够构造出全局最优解

证明思路= 1. 假设存在一个最优解，证明可以修改该解，使其包含当前贪心选择

2. 通过归纳法或构造性替换，展示贪心选择不会破坏全局最优性

2. 最优子结构性质=问题的最优解包含子问题的最优解

证明思路= 1. 假设原问题的最优解由子问题的最优解组合而成

2. 通过递归或归纳法，保证子结构是最优的

☆交换论证= 1. 假设存在某个最优解 $S \neq G$

2. 找到 S 和 G 的第一个差异点

3. 交换后构造新解 S'，S' 接近 G，且仍为最优解

4. 证明 G 与 S 等价或更优

1. Kruskal算法（最小生成树）

- **贪心策略**：按边权从小到大选择边，不形成环。
- **交换论证**：
  1. 假设最优生成树 T 不包含某条贪心选择的边 e。
  2. 将 e 加入 T，形成环。
  3. 删除环中权值最大的边 e'，得到新生成树 T'。
  4. 由于 e 是当前最小边，$w(e) \leq w(e')$，故 T' 权值不劣于 T，证明贪心边必须被包含。

2. Dijkstra算法（单源最短路径）

- **贪心策略**：每次选择距离起点最近的未访问节点。
- **归纳法证明**：
  1. **基例**：起点到自身距离为0，正确。
  2. **归纳假设**：前 k 个节点的最短路径已确定。
  3. **归纳步骤**：第 $k+1$ 个节点通过已确定的最短路径扩展，保证全局最优。

2. 应用示例：活动选择问题

- **问题描述**：选择最多数量的互不重叠活动。
- **贪心策略**：每次选择最早结束的活动。
- **交换论证过程**：
  1. 假设存在一个最优解 S，其中第一个活动 a_j 的结束时间晚于贪心解的第一个活动 a_i。
  2. 将 S 中的 a_j 替换为 a_i，得到新解 S'。
  3. 由于 a_i 结束更早，替换后 S' 仍可行，且活动数量不变，仍为最优解。
  4. 重复此过程，最终 S' 变为贪心解 G，证明 G 是最优的。

动态规划——差分约束问题

差分约束系统

在一个差分约束系统中，线性规划矩阵 A 的每一行包括一个 1 和一个 -1，其他所有项皆为 0。因此，由 $Ax \leqslant b$ 所给出的约束条件变为 m 个涉及 n 个变量的差额限制条件，其中的每个约束条件是如下所示的简单线性不等式：

$$x_j - x_i \leqslant b_k$$

这里 $1 \leqslant i, j \leqslant n$，$i \neq j$，并且 $1 \leqslant k \leqslant m$。

例如，我们考虑寻找一个满足下列条件的 5 维向量 $x=(x_i)$ 的问题：

$$\begin{bmatrix} 1 & -1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} \leqslant \begin{bmatrix} 0 \\ -1 \\ 1 \\ 5 \\ 4 \\ -1 \\ -3 \\ -3 \end{bmatrix}$$

这个问题与寻找满足下列的 8 个差分约束条件的变量 x_1，x_2，x_3，x_4，x_5 的取值的问题等价：

$$x_1 - x_2 \leqslant 0 \tag{24.3}$$

$$x_1 - x_5 \leqslant -1 \tag{24.4}$$

$$x_2 - x_5 \leqslant 1 \tag{24.5}$$

$$x_3 - x_1 \leqslant 5 \tag{24.6}$$

$$x_4 - x_1 \leqslant 4 \tag{24.7}$$

$$x_4 - x_3 \leqslant -1 \tag{24.8}$$

$$x_5 - x_3 \leqslant -3 \tag{24.9}$$

$$x_5 - x_4 \leqslant -3 \tag{24.10}$$

665

这个问题的一个可能答案是 $x=(-5, -3, 0, -1, -4)$，读者可以对每个不等式进行验证来证明该向量确实是一个正确答案。事实上，这个问题的答案有多个。另一个答案是 $x'=(0, 2, 5, 4, 1)$。这两个答案之间存在着关联关系：向量 x' 中每个元素比向量 x 中的对应元素的取值大 5。我们在后面将看到，这种关系并不是巧合。

引理 24.8 设向量 $x=(x_1, x_2, \cdots, x_n)$ 为差分约束系统 $Ax \leqslant b$ 的一个解，设 d 为任意常数，则 $x+d=(x_1+d, x_2+d, \cdots, x_n+d)$ 也是该差分约束系统的一个解。

证明 对于每个 x_i 和 x_j，我们有 $(x_j+d)-(x_i+d)=x_j-x_i$。因此，若向量 x 满足 $Ax \leqslant b$，则向量 $x+d$ 也满足该条件。■

差分约束系统在许多不同的应用里都会出现。例如，未知变量 x_i 可能代表的是事件发生的时间。每个约束条件给出的是在两个事件之间必须间隔的最短时间或最长时间。也许，这些事件代表的是产品装配过程中所必须执行的任务。如果在时刻 x_1 使用一种需要 2 个小时才能风干的粘贴剂材料，则我们需要等到该粘贴剂干了之后才能在时刻 x_2 安装部件，这样，我们就有约束条件 $x_2 \geqslant x_1+2$，或者等价地，$x_1 - x_2 \leqslant -2$。也许，我们可能要求部件必须在粘贴剂涂上后但在粘贴剂半干之前的时间段里安装上。在这种情况下，我们得到一对约束条件：$x_2 \geqslant x_1$ 和 $x_2 \leqslant x_1+1$，而这与差分约束系统 $x_1 - x_2 \leqslant 0$ 和 $x_2 - x_1 \leqslant 1$ 等价。

约束图

我们可以从图论的观点来理解差分约束系统。在一个 $Ax \leqslant b$ 的差分约束系统中，我们将

$m\times n$的线性规划矩阵A看做是一张由n个结点和m条边构成的图的关联矩阵的转置。(请参阅练习22.1-7)。对于$i=1,2,\cdots,n$，图中的每个结点v_i对应n个未知变量x_i中的一个。图中的每条有向边则对应m个不等式中的一个。

正形式化地说，给定差分约束系统$Ax\leqslant b$，其对应的约束图是一个带权重的有向图$G=(V,E)$，这里：

$$V=\{v_0,v_1,\cdots,v_n\}$$

$$E=\{(v_i,v_j):x_j-x_i\leqslant b_k\text{是一个约束条件}\}\cup\{(v_0,v_1),(v_0,v_2),(v_0,v_3),\cdots,(v_0,v_n)\}$$

约束图包含一个额外的结点v_0，用来保证图中至少存在一个结点，从其出发可以到达所有其他的结点。因此，结点集合V由代表每个变量x_i的结点v_i和额外的结点v_0所组成。边集合E包含的是代表每个差分约束的边，再加上边(v_0,v_i)，$i=1,2,\cdots,n$。如果$x_j-x_i\leqslant b_k$是一个差分约束条件，则边(v_i,v_j)的权重为$w(v_i,v_j)=b_k$。所有从结点v_0发出的边的权重为0。图24-8描述的是差分约束系统(24.3)～(24.10)的约束图。

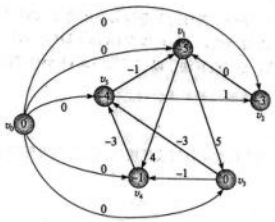


图24-8 差分约束系统(24.3)～(24.10)所对应的约束图。每个结点v_i中的数值是$\delta(v_0,v_i)$的值。该系统的一个可行解是$x=(-5,-3,0,-1,-4)$

下面的定理将证明可以通过在对应的约束图中寻找最短路径权重来找到一个差分约束系统的解。

定理24.9 给定差分约束系统$Ax\leqslant b$，设$G=(V,E)$是该差分约束系统所对应的约束图。如果图G不包含权重为负值的环路，则

$$x=(\delta(v_0,v_1),\delta(v_0,v_2),\delta(v_0,v_3),\cdots,\delta(v_0,v_n)) \tag{24.11}$$

是该系统的一个可行解。如果图G包含权重为负值的环路，则该系统没有可行解。

证明 首先证明如果约束图不包含权重为负值的环路，则式(24.11)给出一个可行解。考虑任意一条边$(v_i,v_j)\in E$，根据三角不等式，$\delta(v_0,v_j)\leqslant\delta(v_0,v_i)+w(v_i,v_j)$，即$\delta(v_0,v_j)-\delta(v_0,v_i)\leqslant w(v_i,v_j)$。因此，如果设$x_i=\delta(v_0,v_i)$和$x_j=\delta(v_0,v_j)$，则$x_i$和$x_j$满足对应边$(v_i,v_j)$的差分约束条件$x_j-x_i\leqslant w(v_i,v_j)$。

现在我们来证明如果约束图包含权重为负值的环路，则差分约束系统没有可行解。不失一般性，设权重为负值的环路为$c=\langle v_1,v_2,\cdots,v_k\rangle$，这里$v_1=v_k$。(结点$v_0$不可能包含在环路$c$上，因为它没有进入的边。)环路$c$对应下面的差分约束条件组：

$$\begin{aligned}x_2-x_1&\leqslant w(v_1,v_2)\\x_3-x_2&\leqslant w(v_2,v_3)\\&\vdots\\x_k-x_{k-1}&\leqslant w(v_{k-2},v_{k-1})\\x_k-x_{k-1}&\leqslant w(v_{k-1},v_k)\end{aligned}$$

我们使用反证法来进行证明。假设向量x有一个满足上述k个不等式的解，则这个解必须同时满足将k个不等式加起来之后形成的新的不等式。如果将不等式的左边进行求和，每个未知变量x_i被加进来一次，被减去一次(记住，$v_1=v_k$意味着$x_1=x_k$)，使得左边的和为0。不等式右面的和为$w(c)$，因此有$0\leqslant w(c)$。但是c是一个权重为负值的环路，因此$w(c)<0$，这样我们得出矛盾：$0\leqslant w(c)<0$。

求解差分约束系统

定理24.9告诉我们，可以使用Bellman-Ford算法来求解差分约束系统。因为约束图包含从源结点v_0到所有其他结点的边，任何权重为负值的环路都可以从结点v_0到达。如果Bellman-Ford算法返回TRUE值，则最短路径权重给出的是该系统的一个可行解。例如，在图24-8中，最短路径权重提供的可行解是$x=(-5,-3,0,-1,-4)$，根据引理24.8，对于任意常数d，$x=(d-5,d-3,d,d-1,d-4)$也是一个可行解。如果Bellman-Ford算法返回FALSE值，则差分约束系统没有可行解。

一个有n个未知变量和m个约束条件的差分约束系统所生成的约束图有$n+1$个结点和$n+m$条边。因此，使用Bellman-Ford算法，我们可以在$O((n+1)(n+m))=O(n^2+nm)$时间内求解该系统。练习24.4-5将要求读者来修改算法，以使其能够在$O(nm)$时间内完成运算，即使m远小于n。

有向无环图$G=(V,E)$的关联矩阵是一个满足下述的$|V|\times|E|$矩阵$B=(b_{ij})$

$$b_{ij}=\begin{cases}-1 & \text{如果边}j\text{从结点}i\text{发出}\\ 1 & \text{如果边}j\text{进入结点}i\end{cases}$$

关联矩阵性质：

1. 秩与连通性：(n为顶点数)

如果G是连通的，则秩为n-1，

如果G有c个连通分量，则秩为n-c。

2. BB^T得到$n\times n$矩阵 $(n=|V|)$

对角线元素=顶点的度(出+入)

e_{ij}=存在i的或j→i的边，

3. B^TB得到$m\times m$矩阵 $(m=|E|)$

e_i和e_j共享起点，$e_{ij}=e_{ji}=-1$

e_i和e_j共享终点，$e_{ij}=e_{ji}=1$

对角线$e_{ii}=2$

动态规划 — 矩阵连乘问题

问题：输入n个矩阵 $A_1 \dots A_n$，记 A_i 的维数是 $p_{i-1} \times p_i$，A_i 和 A_{i+1} 是可乘的

输出最优的计算次序

若A是 $p \times q$ 的矩阵，B是 $q \times r$ 的矩阵，则 $A \times B$ 的代价是 pqr

状态转移方程：

$$m(i,j) = \begin{cases} 0 & i=j \\ \min\limits_{i \le k < j} \{ m(i,k) + m(k+1,j) + p_{i-1}p_k p_j \} & i<j \end{cases}$$

$(A_i \dots A_k)(A_{k+1} \dots A_j)$ 的总计算量是 $A[i:k] + A[k+1:j] + A[i:k] \times A[k:j]$

例：

	A_1	A_2	A_3	A_4	A_5	A_6
	30×35	35×15	15×5	5×10	10×20	20×25
j	1	2	3	4	5	6
i						
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

$A_1 (A_2((A_3 A_4) A_5) A_6))$

一共有 $\frac{1}{2}n(n+1)$ 的总状态数，时间复杂度 $T(n) = \Omega(n^2)$

```
MATRIX-CHAIN-ORDER(p):
  n = p.length - 1  // p[0..n] 定义了 n 个矩阵的维度
  创建两个二维表 m[1..n, 1..n] 和 s[1..n, 1..n]

  // 初始化基础情况：单个矩阵链（长度为1）
  for i = 1 to n:
    m[i][i] = 0

  // l 是子链长度，从 2 到 n
  for l = 2 to n:
    // i 是子链起始点
    for i = 1 to n - l + 1:
      j = i + l - 1  // j 是子链结束点
      m[i][j] = ∞    // 初始化为无穷大
      // 尝试所有可能的分割点 k (i <= k < j)
      for k = i to j - 1:
        // 计算代价
        cost = m[i][k] + m[k+1][j] + p[i-1] * p[k] * p[j]
        // 如果找到更小的代价，更新 m[i][j] 并记录 k
        if cost < m[i][j]:
          m[i][j] = cost
          s[i][j] = k

  return m and s
```

动态规划 — 最优二叉搜索树

问题：给定一组有序关键字 $k_1 \dots k_n$，和对应的搜索概率 $p_1 \dots p_n$，以及 $n+1$ 个虚拟关键字（表示搜索失败区间）的概率 $q_0 \dots q_n$。目标是构建一棵 BST 使搜索代价最小。

动态规划视角：最优子结构，状态转移方程如下：

e_{ij} = 子问题 $k_i \dots k_j$ 的最小期望搜索代价

w_{ij} = 子树 $i \to j$ 的总概率

$$w_{ij} = \sum_{k=i}^{j} p_k + \sum_{k=i-1}^{j} q_k$$

$root_{ij}$ = 最优根节点位置

基础情况：$e_{i(i-1)} = q_{i-1}$，$w_{i(i-1)} = q_{i-1}$

递归情况：$w_{ij} = w_{i(j-1)} + p_j + q_j$

$$e_{ij} = \min_{i \le r \le j} \{ e_{i(r-1)} + e_{(r+1)j} + w_{ij} \}$$

- $e_{i(r-1)}$：左子树代价
- $e_{(r+1)j}$：右子树代价
- w_{ij}：整棵子树的概率和（添加根节点后所有节点深度 +1，代价 + w_{ij}）

```
for l in range(1, n+1)          子树关键字数
    for i in range(1, n-l+2)    起始索引
        j = i + l - 1           结束索引
        e[i][j] = ∞
        w[i][j] = w[i][j-1] + p[j] + q[j]
        for r in range(i, j+1)  根 r
            cost = e[i][r-1] + e[r+1][j] + w[i][j]
            if cost < e[i][j]:
                e[i][j] = cost
                root[i][j] = r
return e[1][n], root
```

时间复杂度 $O(n^3)$

一维动态规划问题：

双数组匹配或单数组自匹配

硬币问题（背包DP）

树状DP

所有结点对的最短路径问题

基于Dijkstra = 使用线性数组实现最小优先队列，算法的运行时间将是 $O(VlV^2+E)) = O(V^3)$

使用二叉堆 = $O(VE\log V)$，有利于稀疏图

使用斐波那契堆 = $O(V^2\log V + VE)$

基于Floyd = $O(V^3E)$，稠密图 $O(V^4)$

1. 自底向上计算最短路径权重 = $\Theta(V^3\log V)$

```
ESP-Basic(L, W)
  n = L.rows // |V|
  let L' = (l'ij) be a new n×n matrix
  for i = 1 to n =
    for j = 1 to n =
      l'ij = ∞
      for k = 1 to n =
        l'ij = min(l'ij, lik + wkj)
  return L'
```

基于矩阵乘法性质进行替换

```
ESP-Martix(A, B)
  n = A.rows
  let C be a new n×n matrix
  for i = 1 to n
    for j = 1 to n
      cij = 0
      for k = 1 to n
        cij = cij + aik · bkj
  return C
```

最终目的是为了得到 $L^{(n-1)}$ = $L^{(1)} = L^{(0)} \cdot W$，$L^{(1)} = W$，因此要递归调用这个算法 n 次

改进算法的执行时间 =

重复平方 = $L^1 = W$ $L^2 = W^2$ $L^4 = (W^2)^2$ … $L^{2^{\lceil \lg(n-1)\rceil}} = W^{2^{\lceil \lg(n-1)\rceil}} = W^{2^{\lceil \lg(n-1)\rceil}} \cdot W^{2^{\lceil \lg(n-1)\rceil}-1}$

得到 $\Theta(n^3\log n)$ 的算法

2. Floyd-Warshall = $\Theta(V^3)$

```
FIDYD-WARSHALL(W)   "尝试经过某个点，判断新路径会不会更短"
  n = W.rows
  D(0) = W
  for k = 1 to n =
    let D(k) = (d(k)ij) be a new n×n martix
    for i = 1 to n = for j = 1 to n =
      d(k)ij = min(d(k-1)ij, d(k-1)ik + d(k-1)kj)
  return D^k
```

记录生成树 = $\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{若 } d_{ij}^{(k-1)} \le d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\ \pi_{kj}^{(k-1)} & \text{若 } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \end{cases}$

传递闭包 = 可达性检查

逻辑词汇的等效替代 =

$t_{ij}^{(0)} = I(i=j \text{ 或 } (i,j)\in E)$

$t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee (t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)})$

初始化 0.2.? 5

找A 0.2.? 5

找B 0.2.9.4

找C 0.2.9.4

找D 0.2.8.4

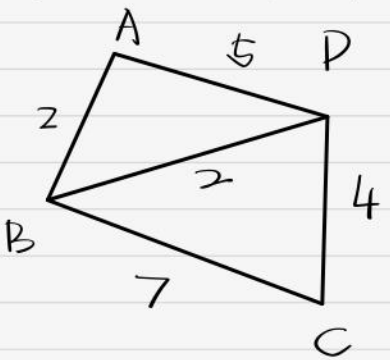
5.2.4.0

5.2.4.0

4.2.4.0

4.2.4.0

4.2.4.0



2.0.7.2

2.0.7.2

2.0.7.2

2.0.7.2

2.0.6.2

?.7.0.4

?.7.0.4

9.7.0.4

9.7.0.4

8.6.0.4

可以认为是从一个小的子图不断扩大，引入更多节点。

3. Johnson 稀疏图 $O(V^2\log V + VE)$

重新赋予权重：如果图 $G=(V,E)$ 中所有的边权重 w 皆为非负值，则通过对每个结点运行一次 Dijkstra 算法来找到所有结点对之间的最短路径，使用斐波那契优化最小优先队列，$O(V^2\lg V + VE)$

如果图 G 包含权重为负值的边，但没有权重为负值的环路，那么只要计算出一组新的非负权重值

下面对图 G 进行预处理，在 $O(VE)$ 的时间内计算出 $\hat{w}$：

1、定理：重新赋予权重并不改变最短路径

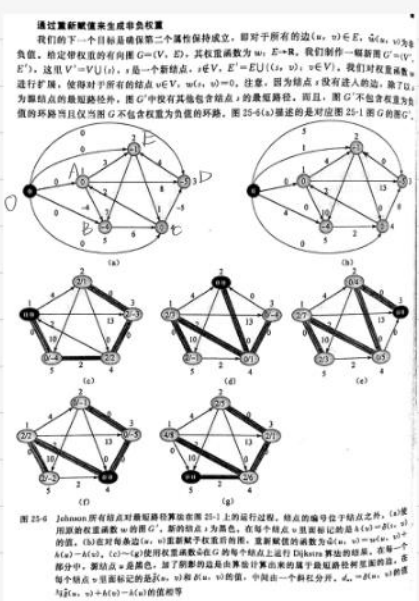
给定带权重的有向图 $G=(V,E)$，其权重函数为 $w: E\to R$，设 $h: V\to R$ 为任意函数，该函数将点映射到实数上。对于每条边 $(u,v)\in E$，定义：$\hat{w}(u,v)=w(u,v)+h(u)-h(v)$

设 $p=\langle v_0,v_1,\dots,v_k\rangle$ 为从结点 v_0 到结点 v_k 的任意一条路径，那么 p 是在使用权重函数 w 时从结点 v_0 到结点 v_k 的一条最短路径，当且仅当 p 是在使用权重函数 $\hat{w}$ 时从结点 v_0 到结点 v_k 的一条最短路径，即 $w(p)=\delta(v_0,v_k)$ 当且仅当 $\hat{w}(p)=\hat{\delta}(v_0,v_k)$。而且，图 G 在使用权重函数 w 时不包含权重为负值的环路，当且仅当 p 在使用权重函数 $\hat{w}$ 也不包括权重为负值的环路。

$$\hat{w}(p)=w(p)+h(v_0)-h(v_k)$$

$$\hat{w}(p)=\sum_{i=1}^{k}\hat{w}(v_{i-1},v_i)=\sum_{i=1}^{k}(w(v_{i-1},v_i)+h(v_{i-1})-h(v_i))=w(p)+h(v_0)-h(v_k)$$

2、预处理：制作一幅新图 $G'=(V',E')$，这里 $V'=V\cup\{s\}$，s 是一个新结点，$s\notin V$，$E'=E\cup\{(s,v),v\in V\}$ 使 $w(s,v)=0$。且因为 s 没有入边，故除了以 s 为源节点的最短路径外，图 G' 中没有其他包含结点 s 的最短路径。而且 G' 和 G 是否有负权环路性质相同。


通过重新赋值来生成非负权重

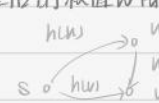
我们的下一个目标是确保第二个属性保持成立，即对于所有的边 $(u,v)\in E$，$\hat{w}(u,v)$ 为非负值。给定带权重的有向图 $G=(V,E)$，其权重函数为 w，$E\to R$。我们制作一幅新图 $G'=(V',E')$，这里 $V'=V\cup\{s\}$，s 是一个新结点，$s\notin V$，$E'=E\cup\{(s,v):v\in V\}$。我们对权重函数 w 进行扩展，使得对于所有的结点 $v\in V$，$w(s,v)=0$。注意，因为结点 s 没有进入的边，除了以 s 为源结点的最短路径外，图 G' 中没有其他包含结点 s 的最短路径，而且，图 G' 不包含有重为负值的环路当且仅当图 G 不包含有重为负值的环路。图 25-6(a)描述的是对应图 25-1 图 G 的图 G'。

图 25-6 Johnson 所有结点对最短路径算法在图 25-1 上的运行过程。结点的编号位于结点之外，(a)使用原始权重函数 w 的图 G'，新的结点 s 为黑色，在每个结点 v 旁面标记的是 $h(v)=\delta(s,v)$ 的值。(b)在对每条边 (u,v) 重新赋予权重后的图，重新赋值的函数为 $\hat{w}(u,v)=w(u,v)+h(u)-h(v)$。(c)~(g)使用权重函数 $\hat{w}$ 在 G 的每个结点上运行 Dijkstra 算法的结果，在每一个部分中，源结点 u 是黑色，加了阴影的边是由算法计算出来的属于最短路径树里面的边。在每个结点 v 里面标记的是 $\hat{\delta}(u,v)$ 和 $\delta(u,v)$ 的值。中间由一个斜杠分开。$d_{uv}=\delta(u,v)$ 的值与 $\hat{\delta}(u,v)+h(v)-h(u)$ 的值相等

以 S 为起点，运行 Bellman-Ford 算法，如果检测到负权边：

将每条边的权值调整为 $w'(u,v)=w(u,v)+h(u)-h(v)$

调整后的权值 $\hat{w}$ 由三角不等式可知一定不小于 0 $w(u,v)\geq h(v)-h(u)$



$\hat{w}=w+h(u)-h(v)$ 是一个回路 iff (u,v) 在最短路上

之后除去 S 节点，再使用 Dijkstra 方法计算

$0\to A=0$

$0\to A\to B=-4$

$0\to C=0$

$0\to C\to D=-5$

$0\to D\to E=-1$

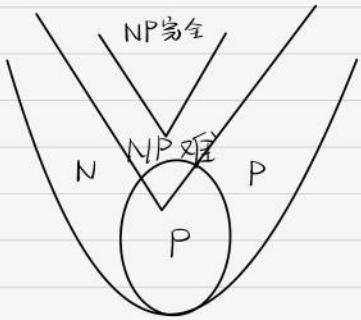
$-0\to B$

有 $0\to A+A\to B+B\to 0\geq 0$

故 $A\to B'=0+(-4)-(-4)=0$

$C\to D'=0+(-5)-(-5)=0$

P、NP、NPC问题



一、问题分类

P问题 = 能在多项式时间内解决

NP问题 = 能在多项式时间内判定

NP难问题 = $\forall Q \in NP,\ Q \le_p P$，则P是NP难问题

可以用多项式时间转化后解决所有NP问题

NP完全问题 = NP难问题中的NP问题

NP完全问题可以在多项式时间内被判定

NP难问题中有停机问题等无法在多项式时间内被判定的问题

二、NP完全性证明：

如果L是一种对于 $\forall L' \in NPC$ 都有 $L' \le_p L$ 的语言，则L是NP难度的。

此时如果 $L \in NP$，则 $L \in NPC$

最基础的NPC问题 = 电路可满足性问题（由∧、∨、¬连接的式子的可满足性）

1. 构造归约函数使对于任意NP问题合法输入x都有 = $x \in L \Leftrightarrow f(x) \in$ 电路可满足性问题

取L的验证算法A，设T(n)表示A对长度为x的输入串的最坏情况下的运行时间，证书长度为O(poly(n))，同时T(n)算法复杂度也为O(poly(n))

2. 算法是个"状态机" = 输入 / 状态数 / 步数是多项式，输出也是多项式

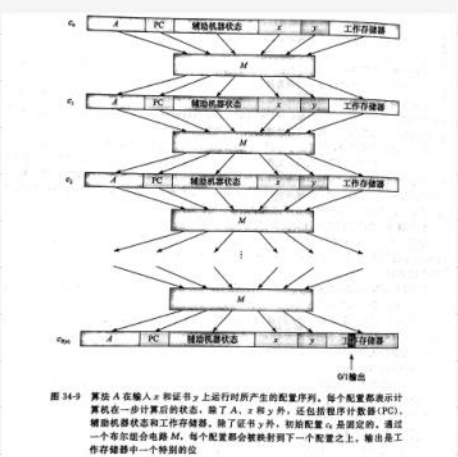


图 34-9 算法 A 在输入 x 和证书 y 上运行时所产生的配置序列。每个配置都表示计算机在一步计算后的状态，除了 A、x 和 y 外，还包括程序计数器（PC）、辅助机器状态和工作存储器。除了证书 y 外，初始配置 c_0 是固定的。通过一个布尔组合电路 M，每个配置都会被映射到下一个配置之上。输出是工作存储器中一个特别的位

我们还需要证明两条性质。第一，必须证明 F 能够正确地计算出归约函数 f，即必须证明 C 是可满足的当且仅当存在一个证书 y，满足 $A(x, y) = 1$。第二，必须证明 F 的运行时间为多项式时间。

为了证明 F 能够正确地计算出归约函数，假设存在一个长度为 $O(n^k)$ 的证书 y，满足 $A(x, y) = 1$，那么，如果把 y 的各位作为 C 的输入，那么 C 的输出为 $C(y) = A(x, y) = 1$。因此，如果有一个证书存在，则 C 是可满足的（电路）。另一方面，假定 C 是可满足的，因此对 C 存在一个输入 y，满足 $C(y) = 1$，据此，可以得到 $A(x, y) = 1$。因此，F 能够正确地计算出一个归约函数。

为了完成此证明过程，仅需证明 F 的运行时间是关于 $n = |x|$ 的多项式时间。首先需要注意的是，表示一个配置所需的位数是关于 n 的多项式。A 的程序本身的规模为常数，与输入 x 的长度无关。输入 x 的长度为 n，证书 y 的长度为 $O(n^k)$。由于算法至多运行 $O(n^k)$ 步，所以 A 所要求的工作存储器总量也是 n 的多项式。（假定该存储器单元是连续的；练习 34.3-5 要求读者把证明扩展到下列情况：A 所存取的存储单元散布于存储器的一个更大的范围内，对每个输入 x，其特定的散布方式也可能不同。）

实现计算机硬件的组合电路 M 的规模是关于配置的长度的多项式，即为 $O(n^k)$ 的多项式，因而也是关于 n 的多项式。（这个电路的大部分实现了存储系统的逻辑。）电路 C 至多由 $t = O(n^k)$ 个 M 的副本组成，因此其规模是关于 n 的多项式。由于构造过程的每一步都需要多项式时间，所以用归约算法 F 可以在多项式时间内完成从 x 构造电路 C 的过程。

综上所述，因为语言 CIRCUIT-SAT 至少与 NP 中的任意语言具有同样的难度，并且又因为它是属于 NP 的，所以它是 NP 完全的。

定理 34.7 电路可满足性问题是 NP 完全的。

证明 根据引理 34.5 和引理 34.6，以及 NP 完全性的定义，可以直接推得结论。 ■

常见的转化关系：

1. 布尔 ↔ 图：文字 ↔ 点　能同时满足的文字用边相连：CNF-SAT问题 ↔ 团问题
2. 补图：团 ↔ 顶点覆盖　G有k的团 ↔ $\bar{G}$有一个规模为$|V|-k$的顶点覆盖
3. 边 ↔ 图结构：顶点覆盖 ↔ 哈密顿回路

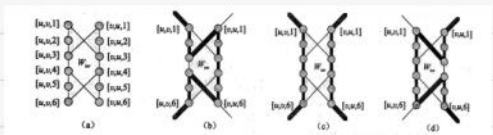


图 34-16　在将顶点覆盖问题归约为哈密顿回路的过程中所用到的附件图。图 G 的一条边 (u, v) 对应于归约过程所产生的图 G' 中的附件 W_{uv}。(a)附件图，其中的每个顶点都加上了标记。(b)～(d)加上了阴影的路径是通过附件图且包含所有顶点的仅有的可能路径，假设从该附件图到 G' 的其余部分的唯一连接是通过顶点 $[u, v, 1]$、$[u, v, 6]$、$[v, u, 1]$ 和 $[v, u, 1]$ 完成的

将图中的每一条边都映射成"附件图"
附件图有特殊的性质，这里使用如右所示的附件图，图的特点是：
① 有限的覆盖所有点的路径
② 不存在分别包含两点"出口"和"入口"的路径使路径的并包含所有的顶点

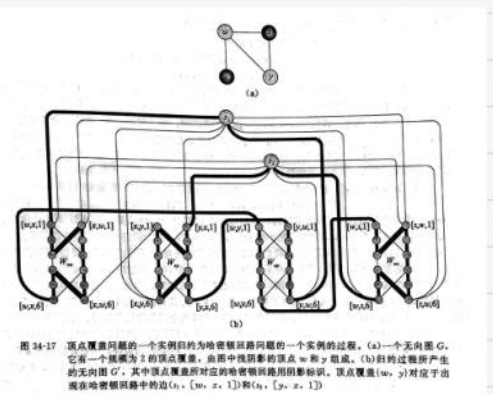


图 34-17　顶点覆盖问题的一个实例归约为哈密顿回路问题的一个实例的过程。(a)一个无向图 G，它有一个规模为 2 的顶点覆盖，由图中浅阴影的顶点 w 和 y 组成。(b)归约过程所产生的无向图 G'，其中顶点覆盖所对应的哈密顿回路用阴影标识。顶点覆盖 (w, y) 对应于出现在哈密顿回路中的边 $\langle x_1, [w, x, 1]\rangle$ 和 $\langle x_2, [y, x, 1]\rangle$

1. 假设$G'=(V', E')$中包含一个哈密顿回路$C \subseteq E'$，则集合$V^* = \{u \in V : \langle s_j, [u, u^{(i)}, i]\rangle \in C, 1 \le j \le k\}$是$G$的一个顶点覆盖
2. 假设$G=(V, E)$中有一个规模为k的顶点覆盖V^*，$V^* \subseteq V$，设$V^* = \{u_1, \ldots u_k\}$，通过为每个顶点$u_j \in V^*$包含对应边，就可以形成哈密顿回路

4. int ↔ bool：哈密顿回路 ↔ 旅行商问题
5. set ↔ bool：子集和问题 ↔ 3-CNF-SAT　使用近似于"位图"的数据表示方法记录式和文字的信息。



图 34-19　从 3-CNF-SAT 到 SUBSET-SUM 的归约。3-CNF 的公式为 $\phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$，其中 $C_1 = (x_1 \vee \neg x_2 \vee \neg x_3)$，$C_2 = (\neg x_1 \vee \neg x_2 \vee \neg x_3)$，$C_3 = (\neg x_1 \vee \neg x_2 \vee x_3)$，$C_4 = (x_1 \vee x_2 \vee x_3)$。$\phi$ 的一个可满足性赋值为 $x_1 = 0$，$x_2 = 0$，$x_3 = 1$。归约过程所生的集合 S 包含了一些十进制数，按照从上往下的顺序，$S = \{1001001, 1000110, 100001, 101110, 10011, 11100, 1000, 2000, 100, 200, 10, 20, 1, 2\}$，目标 t 为 1114444。子集 $S' \subseteq S$ 在图中用浅阴影表示，它包含了 v'_1、v'_2 和 v'_3，对应于可满足性赋值。它还包含了检验变量 s_1、s'_1、s_2、s'_2、s_3 和 s'_3，以便表示由 C_1 到 C_4 所标记的数位上达到目标值 4

高位记录对应文字编号
低位记录对应式子